

基于 Python 的电商平台销售数据分析

摘要

电子商务行业的快速发展使得数据驱动决策成为企业精细化运营的核心手段。本文基于 Kaggle 公开的电商平台销售数据集，运用 Python 及其数据分析生态，从销售趋势、品类结构、地域表现、用户行为及时间序列预测等多个维度对平台运营状况进行了系统性分析。

在**销售趋势**方面，平台整体销售额与利润呈平稳波动，季节性特征显著，7 月与 5 月为天然旺季，1 月为明显淡季，峰谷差达 30% 以上。时间序列建模中，ARIMA(1, 1, 1) 在测试集上表现优于 ETS，MAE 为 182.5 万，预测未来半年月销售额将稳定在 2,250 万元左右，但 2025 年 9 月出现的 27.7% 预测偏差提示可能存在模型未能捕捉到的外部因素干扰。

品类分析表明，平台 HHI 指数为 0.1003，属于高度分散的综合性平台，无单一品类依赖风险。其中 Furniture 品类为“利润明星”，利润率超 15% 且销售额居前；Books 与 Groceries 属薄利品类，适合引流。折扣敏感度分析显示多数品类折扣对销量拉动几乎无效，建议平台减少无差别打折。

地域方面，各区域销售额与利润存在统计显著差异但幅度不大，北方贡献最高，南方相对偏低，但整体结构均衡。城市层级分析识别出明显的梯队分化，第一梯队（Bangalore、Lucknow）竞争胶着，与第二梯队存在断层，需差异化配置资源。

用户行为层面，平台复购率仅为 3.2%，单次客户占比 96.8%，属典型的拉新驱动型业务。客户价值分层显示，28.1% 的高价值客户贡献呈断层式领先，超过超高价值与中价值客户总和 320 万元，而 40.3% 的中低及低价值客户仅贡献 11.9% 的销售额。引入购买频次进行二维交叉分析后发现：购买 3 次的客户客单价中位数较单次客户提升约 75%，复购行为实质上是筛选高支付能力客户的有效信号，而非单纯的购买次数积累。

综合而言，该平台是一家品类结构健康、地域分布均衡、但高度依赖拉新驱动的大型综合性电商平台。建议未来运营重点聚焦于：（1）减少无差别折扣，转向品类差异化的定价与权益策略，对高利润品类强化保护，对薄利引流品类控制投入；（2）建立高价值客户专属维护机制，将其按购买频次拆分为“一次性大单型”与“复购积累型”分类运营，防止核心用户流失；（3）以城市梯队为依据进行资源分层配置，第一梯队维持优势并引入良性竞争，第二梯队集中资源扶持 1-2 个城市突破断层；（4）深入排查 2025 年 9 月销售异常原因；（5）将复购行为视为客户质量筛选工具，聚焦推动客户完成第 2 次购买，而非单纯追求复购率数字。

关键词：电商数据分析；Python；销售额预测；用户分层；时间序列分析

前言

电子商务行业的快速发展使得线上交易规模持续扩大，网络购物已成为居民消费的重要渠道。与此同时，电商平台的运营复杂度也在不断提升——品类管理、促销定价、区域资源配置、用户增长与留存等问题相互交织，传统的经验驱动决策方式逐渐难以适应多变的竞争环境。如何从交易数据中提取有效信息，支撑品类优化、促销策略调整、区域资源分配及用户运营决策，是平台运营方面临的实际问题。

本报告结构安排如下：

- 第一部分 数据预处理：说明数据来源、清洗过程、异常值处理及核心指标的相关性分析，为后续建模提供干净的数据基础。
- 第二部分 销售趋势分析：从整体走势、季节性规律、时间序列预测三个层面，回答“平台销售表现如何、未来怎么走”的问题。
- 第三部分 商品品类分析：从品类盈利能力、集中度（HHI 指数）、折扣敏感度三个角度，评估平台品类结构的健康度与优化空间。
- 第四部分 地区表现分析：从区域差异检验（ANOVA）、城市梯队划分、品类偏好下钻三个层次，识别高价值区域与断层城市。
- 第五部分 用户行为分析：从复购情况、客户价值分层（RFM 替代框架）、支付方式三个维度，刻画用户结构与行为模式。
- 第六部分 结论与建议：汇总核心发现，形成可落地的运营策略建议。

一、数据预处理

1.1 数据获取

从 Kaggle 公开数据库获取一个仿真模拟的在线商店电商销售样本 Ecommerce_Sales_Data_2024_2025，该数据库符合 CC0: Public Domain 许可，涵盖了 2024 到 2025 年的订单数据。

利用 Python Pandas 导入数据集。对数据进行初步的观察可以发现，该数据集具有 5000 条样本，包含 14 项如订单 ID、订单时间、产品名称、支付方式等的详细信息。

1.2 缺失值处理

对数据集进行观察后并未发现缺失值。

1.3 重复值处理

对数据集进行观察后发现数据集数据不存在重复值，无需剔除。

1.4 异常值处理

对数据集进行描述性统计分析结果如表 1 所示。可以看出该平台具有高客单价高利润的特征，每一单订购商品的数量最多仅为 5 个，而平均下单商品数量为 3 个，折扣从不打折到 8 折，平均来看总体是 9 折的状态。

发现在 Unit Price（商品单价）以及 Profit（利润）中存在异常数据。Unit Price（商品单价）的最大值为 79998 元，在均值为 39760.905 元、中位值为 39459.5 元的情况下，最小值仅为 222 元；Profit 最大值为 89688.44 元，均值为 15941.747 元，中位值为 11108.525 元的情况下最小值仅有 19.12 元，与该数据集整体的高单价高利润情况存在显著差异。

表 1 Ecommerce_Sales_Data_2024_2025 数据集描述性统计

	Order ID	Quantity	Unit Price	Discount	Sales	Profit
count	5000.000	5000.000	5000.000	5000.000	5000.000	5000.000
mean	12500.500	2.993	39760.905	10.051	106733.205	15941.747
std	1443.520	1.413	22831.784	7.085	85108.208	14897.685
min	10001.000	1.000	222.000	0.000	264.100	19.120
0.250	11250.750	2.000	20312.250	5.000	39766.538	4892.295
0.500	12500.500	3.000	39459.500	10.000	83080.325	11108.525
0.750	13750.250	4.000	59721.750	15.000	156968.588	22467.988
max	15000.000	5.000	79998.000	20.000	398485.000	89688.440

对这两项数据进行提取并判断该数据是否需要剔除。
首先分析商品单价最低的数据记录。如下表所示。

表 2 商品单价最低记录

Order ID	Unit Price	Discount	Sales	Profit	Category
13923	222	15	566	97.74	Toys

查询可得，该记录对应的商品类型为玩具，折扣为 15%，销售额为 566，利润为 97.74，为一条正常的促销数据，说明低单价的玩具在打折促销时仍然能够获得利润。故不予剔除。

再分析利润最低的数据记录，如下表所示。

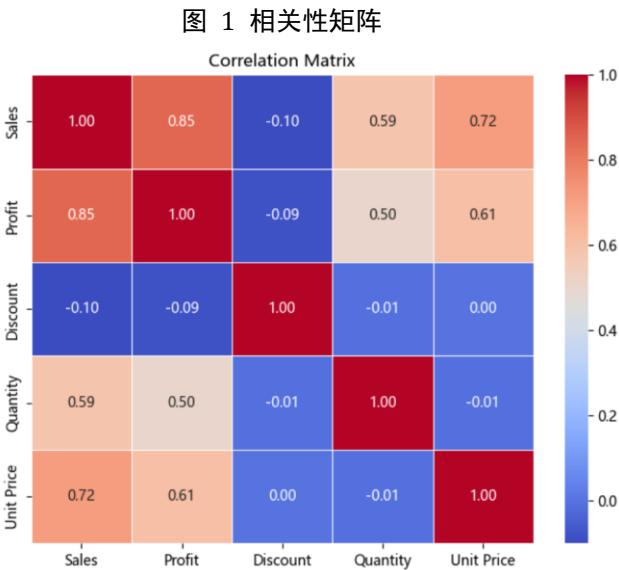
表 3 利润最低记录

Order ID	Unit Price	Discount	Sales	Profit	Category	Product Name
13066	278	5	264	19.12	Furniture	Bed

由表可得，该记录对应的商品类型为家具，商品名称为床，折扣为 5%，销售额 264，客单价仅为 278 元。一张床的正常售价和利润不应该如此低，在无法确认该数据是否是录入错误的情况下，可以认为该商品有概率是样品床或清仓处理床，以清仓价销售只求回本。在总体销售统计中可以保留该数据，但涉及品类客单价、利润率等问题时应予剔除。

1.5 相关性分析

对销售额、利润、折扣、利润、商品单价这五个变量进行相关性分析，得到相关性矩阵如下图。



由图可知，针对 Sales（销售额）而言，其与 Profit（利润）之间的相关系数为 0.85，

强正相关，代表销售额越高，利润相应越高，符合规模效应，即规模化能带来利润增长。与 Unit Price（商品单价）的相关系数为 0.72，强正相关，表明高单价商品能贡献更多的销售额，而不是靠薄利多销的策略获取利润。与 Quantity（销量）之间的相关系数为 0.59，正相关，但小于与单价的相关系数，说明销量的提升能够一定程度上带动销售额的提升，但提升效果不如单价显著。值得注意的是，与 Discount（折扣）之间的相关系数为-0.10，极弱的负相关，说明折扣对销售额几乎没有拉动作用，此情形下采取打折策略并不能有效提升销售额。

针对 Profit（利润）而言，其与 Unit Price（单价）的相关系数为 0.61，说明高单价的商品相应具有更强的获利能力，而与 Quantity（销量）的相关系数为 0.5，说明销量的增加也能带来利润，但获利效果不如提高单价明显。同样地，利润与 Discount（折扣）之间的相关系数为-0.09，折扣对利润几乎没有影响，表明在此情况下打折促销对利润提升没有明显作用。

针对 Discount（折扣）而言，其与销量的相关系数为-0.01，说明打折对销量的影响极小，而与 Unit Price（单价）的相关系数为 0，说明无论单价高低，打折力度都是一致的。

综合而言，该电商平台商家采取打折促销的手段对销量的提升、利润的提升均没有明显的帮助，且无论是高单价的商家还是低单价的商家，采取打折促销手段的效果都差不多。若是想提升销售额以及利润，商家应当从提升商品单价（如销售更贵的商品、引导客户购买高单价商品）入手，而不是盲目打折。

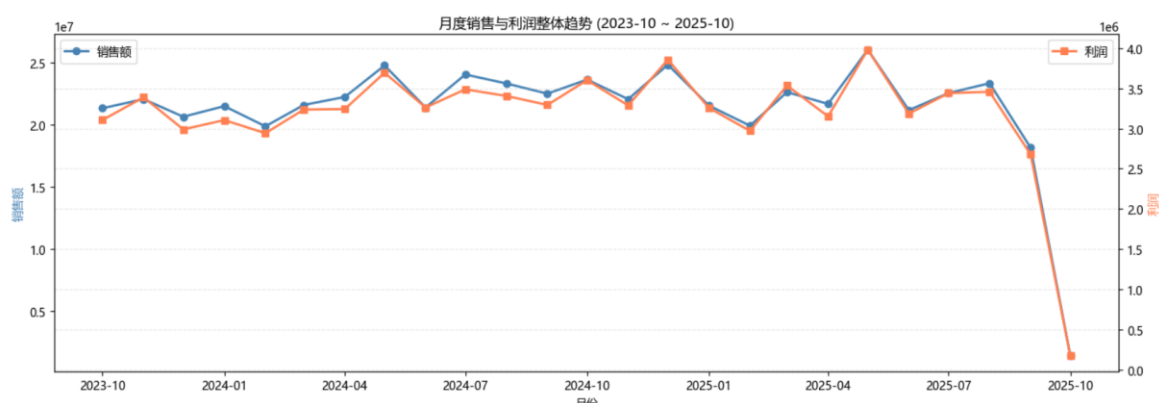
对于平台而言，在策划促销活动时应当抛弃“打折拉动销量”的观念，而应该重点考虑广告、引流等其他手段拉动销量。

二、销售趋势分析

2.1 整体销售趋势分析

电商平台销售趋势通过数据中的 Order Date（下单日期）进行展现，对数据中各个月的销量以及利润进行时间上的可视化，如下图所示。

图 2 月度销售与利润整体趋势



首先对表中 2025-10 的数据点进行说明。通过数据观察可知，该数据集包含内时间跨度为 2023-10-4 至 2025-10-03 的数据，完整月份为 24 个月，2025 年 10 月仅有 3 天数据，因此总利润与总销量会明显低于其他月份。

由图 2 可以看出，整体而言销售趋势呈现平稳趋势，2023 年 10 月总销售额约为 2100 万，总利润约为 310 万，2025 年 9 月总销售额约为 1800 万，总利润约为 270 万，虽有一定程度下降但整体趋势较为稳定。

图 2 可以看出，2024 年的 5 月、12 月以及 2025 年的 5 月呈现明显的高涨趋势，对应了 5 月的国际劳动节、国际母亲节以及年末的黑五促销、圣诞季，判断销量季节性变化有助于商家合理分配库存，做好仓储规划以及运力分配。

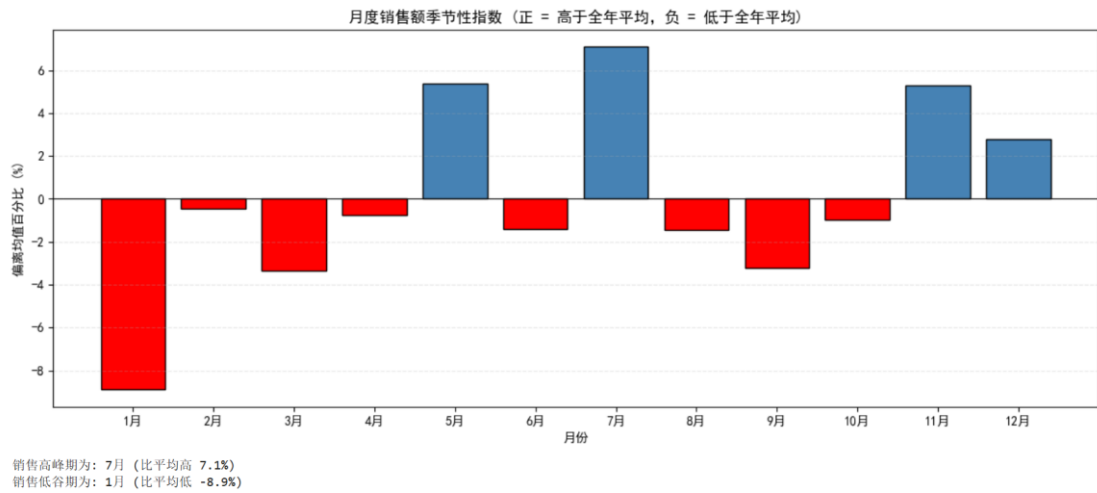
从峰谷值来看，销量额峰值出现在 2025 年 5 月，超过了 2500 万，次峰值为 2024 年 12 月，接近 2500 万，谷值则为 2025 年 9 月，约为 1800 万，次谷值则出现在 2024 年 2 月，约为 2000 万。相应的，总利润的峰值为 2025 年 5 月的 390 万，次峰值为 2024 年 12 月的 380 万；谷值为 2025 年 9 月的 260 万，次谷值为 2024 年 2 月的 290 万。总销售额的峰谷差为 30.29%，总利润的峰谷差为 32.72%，更明确地说明了销售存在着旺季与淡季的差异且相对显著，而利润的波动比销售额更加明显，说明淡季有可能不仅卖得少且利润更薄。

2.2 季节性销售趋势分析

对一年各季度进行进一步分析，判断是否存在更久的淡旺季区分。

由图 3 可以看出，5 月、7 月、11-12 月的销售额高于全年平均销售额，其中 7 月的销售额最高，比平均高了 7.1%，5 月则是第二，比平均值高 5.5%，说明 7 月是天然的旺季，同时 5 月的销售额可能受其他因素影响有所上升，而其中的 6 月虽然相较于全年平均有所不足，但也具有较高的销售额，因而建议商家在 5-7 月、11-12 月这两个旺季进行库存的提前规划；同时 1 月-4 月为销售的淡季，尤其是 1 月，比全年平均低了 8.9%，在该月应当主动策划促销活动以带动销量增长。

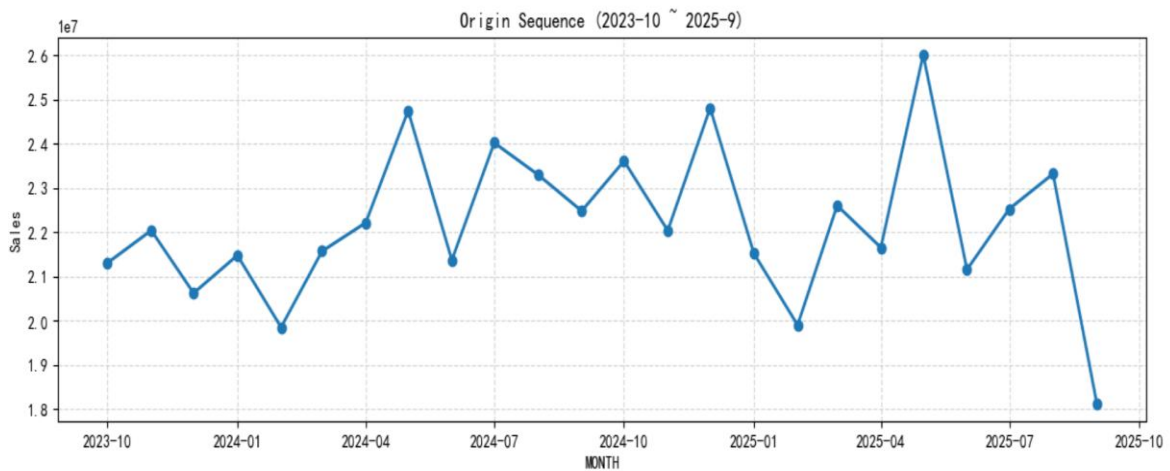
图 3 季节性指数分析



2.3 时间序列预测

有效分析历史销量数据与未来销售量之间的关系能够帮助企业制定重大销售战略，以 ARIMA 模型与 ETS 时间平滑模型建立预测模型得到销售量预测值与销售记录的对应关系，对未来的销量进行预测。

图 4 时间序列原始数据



取前 18 个月作为训练集，后 6 个月为测试集，训练集占 75%，测试集占 25%

2.3.1 ETS 时间序列

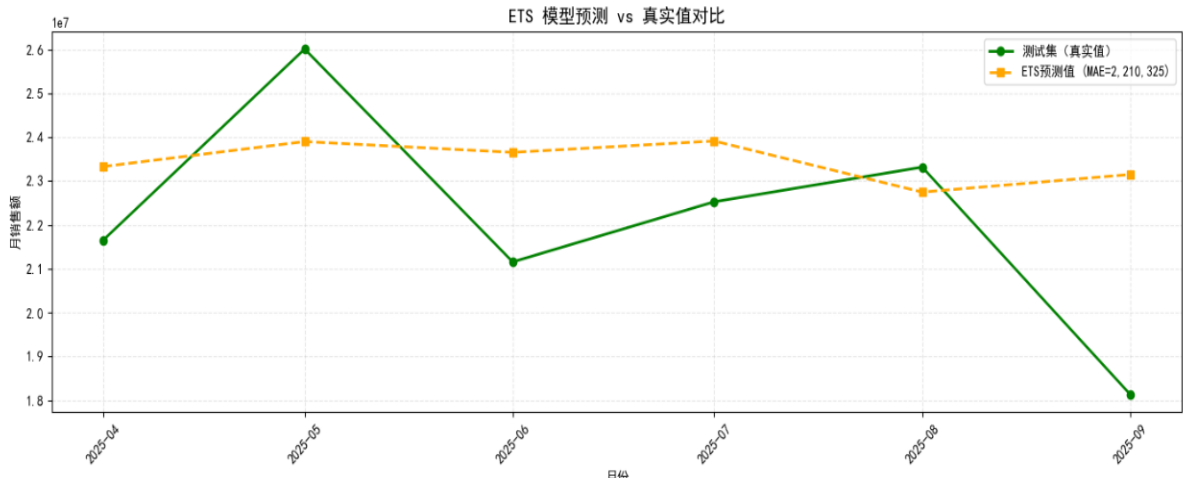
ETS 模型（Error-Trend-Seasonality Model）是一种常用的时间序列预测方法，适用于包含误差、趋势和季节性成分的数据。

使用 ETS 模型对销售量进行预测，并对及结果进行误差分析，得到结果如表 4、图 5 所示。

表 4 ETS 模型统计误差

MAE	RMSE
2, 210, 325. 40	2, 612, 218. 03

图 5 ETS 模型预测与真实值对比



由图 5 可看出，模型预测值与真实值之间存在明显差异且趋势走向不一致，对偏差进行计算，得到表 6。

表 5 ETS 偏差分析

	真实值	预测值	差额	误差
2025-04:	21, 653, 818,	23, 331, 163,	1, 677, 345	7. 70%
2025-05:	26, 010, 929,	23, 900, 387,	-2, 110, 541	-8. 10%
2025-06:	21, 155, 496,	23, 653, 877,	2, 498, 381	11. 80%
2025-07:	22, 526, 568,	23, 913, 898,	1, 387, 331	6. 20%
2025-08:	23, 317, 916,	22, 747, 788,	-570, 128	-2. 40%
2025-09:	18, 131, 497,	23, 149, 722,	5, 018, 225	27. 70%

由表 5 可看出，模型低估了 5 月的销售爆发力，同时对 9 月的预测远高于实际。可能有如下几点可能：

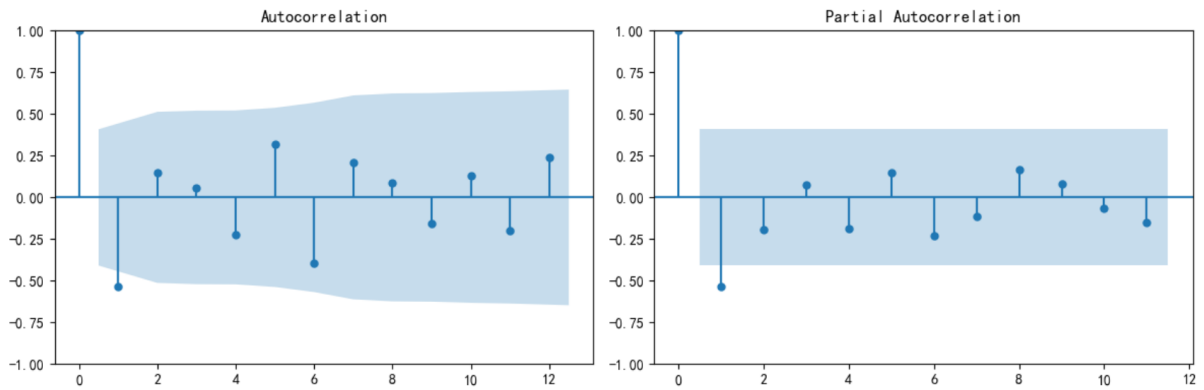
1. 相较于其他月份，9 月缺少合适的促销活动，导致销售量未如预期。
2. 9 月有一定可能性出现畅销品缺货等情况导致销售量下滑。
3. 9 月时其他电商平台加大促销力度导致客户分流，从而使销售量下滑。

2.3.2 ARIMA 时间序列

ARIMA，即自回归积分滑动平均模型（Autoregressive Integrated Moving Average Model），是时间序列分析中最常用的模型之一。它结合了自回归（AR）和滑动平均（MA）两种方法，并加入了差分（I）的概念，以使非平稳时间序列数据变得平稳，从而进行有效的预测。

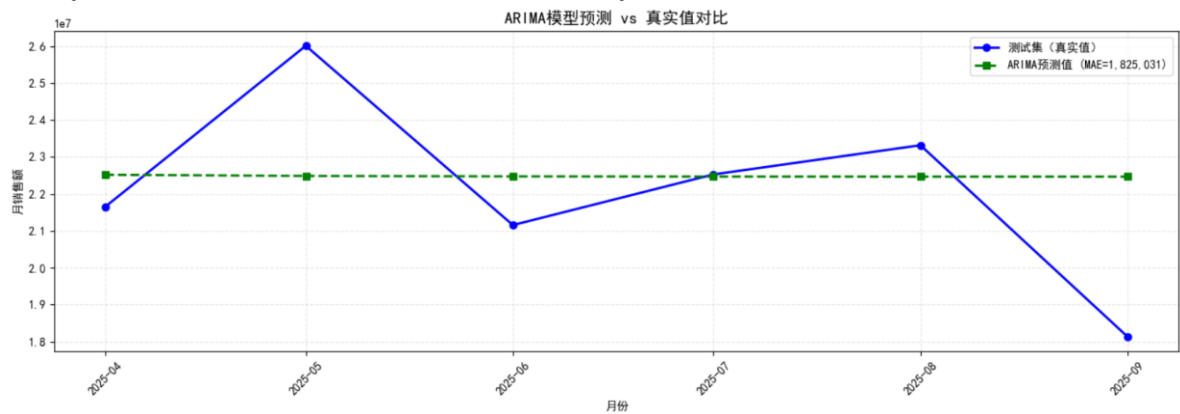
首先对数据进行预处理确保时间连续性，对时间序列模型进行参数估计，绘制序列差分后 ACF 图与 PACF 图，结果如图 6 所示，得到最佳参数为（1, 1, 1）。

图 6 时间序列差分后的 ACF 图与 PACF 图



使用该参数进行时间序列模型预测，得到预测结果与真实值的对比如下图所示。

图 7 ARIMA 模型预测值与真实值对比



计算偏差得到表 6。

	真实值	预测值	差额	误差
2025-04:	21,653,818	22,517,837	864,019	4.00%
2025-05:	26,010,929	22,486,060	-3,524,869	-13.60%
2025-06:	21,155,496	22,474,201	1,318,705	6.20%
2025-07:	22,526,568	22,469,775	-56,792	-0.30%
2025-08:	23,317,916	22,468,124	-849,792	-3.60%
2025-09:	18,131,497	22,467,508	4,336,011	23.90%

由表 6 可以看出，ARIMA 模型同样低估了 5 月的销售爆发力，高估了 9 月的销售低迷现象，同时也说明 9 月销售量的下滑并非季节性因素影响而是存在其他因素干扰，综合而言，选择 ARIMA 模型作为最优模型，理由如下：

1. ETS 模型的 MAE 为 2,210,325.40，ARIMA 模型的 MAE 为 1,825,031.36，ARIMA 模型具有更低的 MAE。
2. ARIMA 模型的预测准确度相较于 ETS 更高，某些点位的偏差仅为 0.3%。
3. ARIMA 模型的预测值更稳定。

三、商品品类分析

3.1 商品品类对销售额等指标的影响

正确选择销售商品的种类，定位销售方向，同样是企业扩大市场、提升销售业绩的决策重点。

对数据集进行分析，汇总得到总销售额品类前十以及总利润品类前十、利润率最高的品类前十，如图及表 6 所示。

图 8 各品类总销售额排行

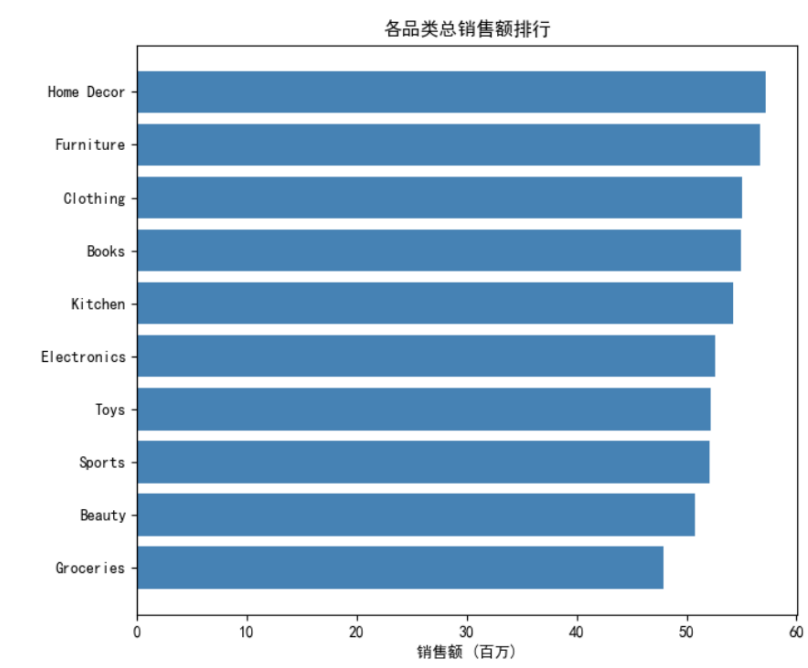


图 9 各品类利润排行

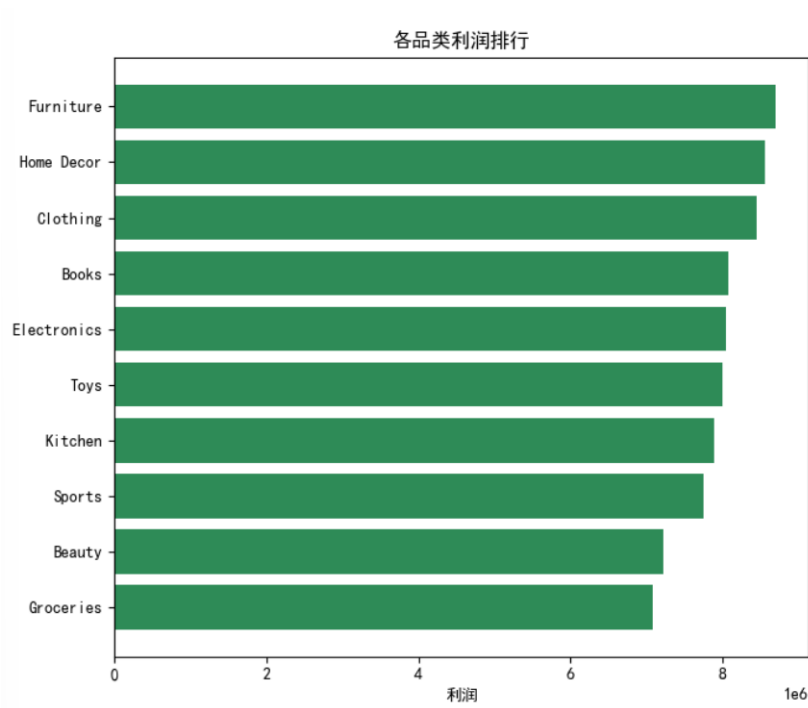


表 6 利润率 TOP10 品类

Category	Sales	Profit	Profit Margin
Furniture	56647187.9	8693087	15.346017
Clothing	55053908.3	8445750	15.340873
Electronics	52587883.95	8042134	15.29275
Toys	52227366.45	7986871	15.292503
Home Décor	57233222.35	8556847	14.950839
Sports	52069397.25	7739430	14.863683
Groceries	47883103.15	7075333	14.776262
Books	54932643	8076273	14.702139
Kitchen	54227902.3	7879573	14.530478
Beauty	50803409.7	7213436	14.198724

由图 8-9 以及表 6 可以看出，Furniture、Clothing 和 Electronic 这三个品类具有最高的利润率，可以称为“印钞机”，尤其是 Furniture，同时具有高销量、高利润及高利润率特征，卖得多且利润率高，商家在选择品类进行上架时可作为核心品类，平台内部的运营在选品时可着重考虑，推流时也可以倾斜一部分资源，保障库存供应。

此外，值得关注的还有 Book 品类，虽然销售总额和总利润都是排名第四，但利润率排倒数第三，说明该品类具有一定的“薄利多销”模式，适合作为引流品类，但不能当做主要的收益来源。

最后，Groceries 品类具有最低的销售额和最低的利润，利润率也是倒数，同样的，Beauty 品类的整体排行都很低，表现中规中矩，为了保证一站式购物的需求应当保留，因为它们也不是完全不盈利，只是表现没有其他的品类好。

3.2 品类集中度分析

赫芬达尔—赫希曼指数（Herfindahl-Hirschman Index, HHI，简称赫芬达尔指数）是衡量产业集中度的综合指标，通过对行业内各企业总收入或总资产百分比的平方和计算得出，数值越大表明市场集中度越高。通过计算该指数，可有效判断平台是否属于“百花齐放”还是少数品牌垄断。计算结果如表所示。

表 7 集中度分析

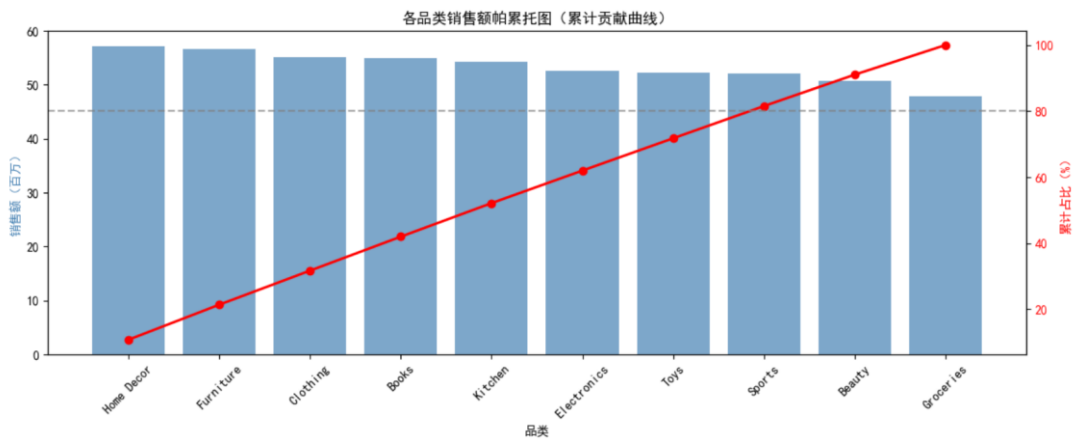
HHI 指数	0.1003
前 1 大品类累计占比	10.70%
前 3 大品类累计占比	31.70%
前 5 大品类累计占比	52.10%

由表 7 可得，HHI 指数为 0.1003，属于品类分散的平台，结构健康，不存在商品品类单一的风险。且前 5 大品类占比为 52.1%，品类高度分散，为综合性平台。且第

一大品类的占比仅为 10.7%，头部品类并未形成压倒性优势，各品类高度竞争，没有哪个品类垄断市场。前五大品类累计占比 52.1%，哪怕是尾部品类也有 47.9% 的贡献率，更说明了该平台呈现“去中心化”的特点，没有以哪个独特品类为卖点，也不只依靠某几个品类生存。

由图 10 不难看出，各品类的贡献呈现单调增长趋势，而不是传统帕累托曲线会出现的 L 型（陡峭）曲线或是 S 型（平缓）曲线，说明各品类对平台的贡献相对均衡，从平台整体结构来看，在进行商家选择以及流量推荐时不必押注在某个单一品类上，也就是说没有“把鸡蛋放在一个篮子里”的风险。即使平台根据品类销量结论对某几个品类进行资源倾斜，也不会有“过度集中”带来的结构性风险，哪怕这几个品类出现波动，平台整体仍然能够平稳运行。

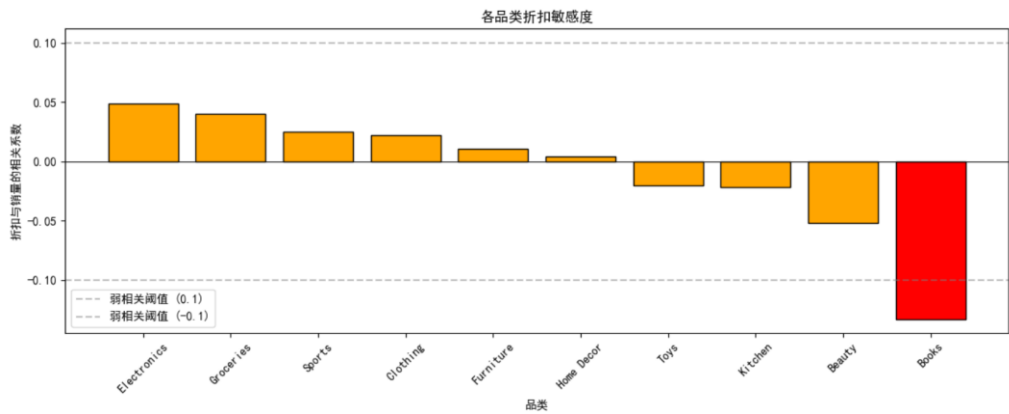
图 10 各品类累计贡献曲线



3.3 品类折扣敏感分析

不同品类商品是否需要打折，打折效果如何对策划平台促销活动、商家制定营销策略同样具有重要意义。通过计算各个品类的折扣-销量相关系数判断哪些品类打折促销可以提升销量，哪些品类打折等于白送钱。结果如图 10 所示。

图 11 各品类折扣敏感度



由上图可知，除 Book 品类，其他品类打折几乎不影响销量，而 Book 品类的相关系数为-0.13，为负敏感类型，打折越多卖得越少，建议少使用打折促销的策略促进销量。

四、地区表现分析

4.1 整体区域分析

根据区域贡献销售量以及利润回报可以得到针对各区域用户偏好的营销手段调整，对吸引新用户、提高复购率等均具有一定的参考作用。

4.1.1 各地区表现分析

根据数据集不难发现，Region 标签下的分类为东南西北四个大区，对这四个区域的销售额、利润、销量等进行汇总，得到表 8。

表 8 各地区表现汇总

Region	Sales	Profit	Order ID	Quantity	Profit_Margin
North	143,578,246.10	21,343,004.33	1288	3949	14.87
East	135,811,637.95	20,532,558.12	1256	3823	15.12
West	131,045,973.35	19,580,123.14	1241	3685	14.94
South	123,230,166.95	18,253,049.32	1215	3506	14.81
AVERAGE	133,416,506.09	19,927,183.73	1,250.00	3,740.75	14.94
Relative Range (%)	15.25	15.51	5.84	11.84	2.08

由表 8 可知，北方地区销售额、利润及销售量均是最高，分别为 14357.82 万，2134.30 万和 3949 单，但利润率排名第三，14.87%，比利润率第一的东部低了 0.25 个百分点。而南方地区的销量、销售额、利润以及利润率皆为最低。

但各区域之间的利润率的差距不是特别显著，最大值与最小值之间相差 0.31%，相对极差为 2.08%。销售额、利润以及销量的相对极差分别为 15.25%、15.51% 和 11.84%。

为进一步判断各区域之间的销售额、利润是否存在显著差异，首先进行方差分析。结果如表所示。

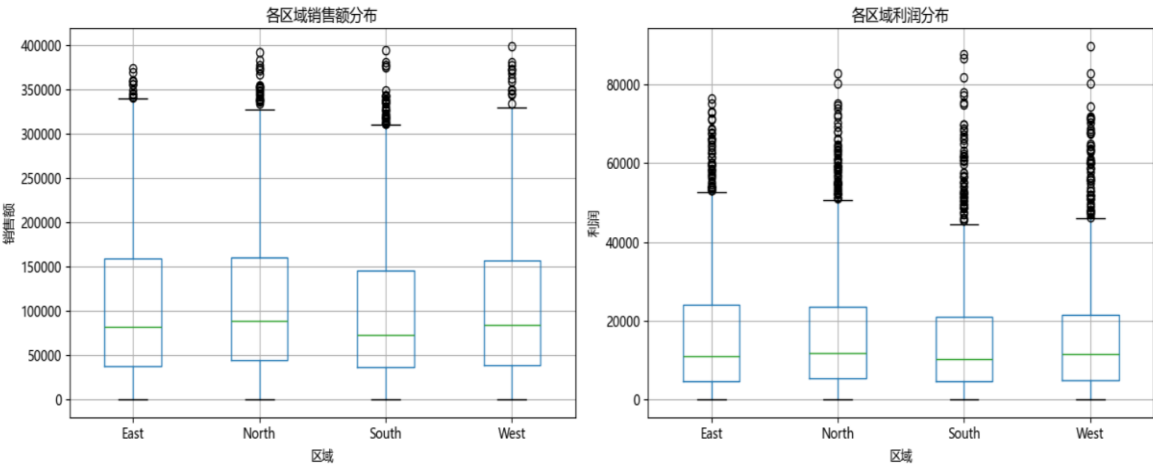
表 9 各区域销售额、利润方差分析

	F 统计量	p 值
Sales	3.0985	0.0256989
Profit	2.6687	0.0460036

不难得出各区域间的总销售额和总利润存在显著差异，但 p 值接近临界水平，说明该差异虽然真实存在但并不属于地方性压倒趋势，也就是说，各区域之间旗鼓相当但排名略分先后。

绘制分组箱线图，如图 12。

图 12 各区域销售额与利润分组箱线图



首先对上图中左图进行分析。同样地，各区域在销售额上的平均水平差不多，且可以看出各个箱体的重叠程度很高，分位数的值也基本相近，因而从整体上看各个区域之间的销售额不存在特别明显的差异，而且连分布趋势都差不多。只是在最大值上存在明显差异，East（东部）最高，接近 350000，而各个区域均存在须线上方的异常值，East（东部）区域异常值相对少而集中，而 North（北部）、South（南部）的异常值更多且分布更广，说明各区域之间的客户质量分布不均，且高净值的客户在地理上存在聚集。

对利润分布进行分析。同样地，各区域之间的利润分布趋势相似，只是在最大值上存在差异。由异常值仍然可以得出与前段相同的结论，各区域的客户质量存在差异，有的区域高净值客户集中。

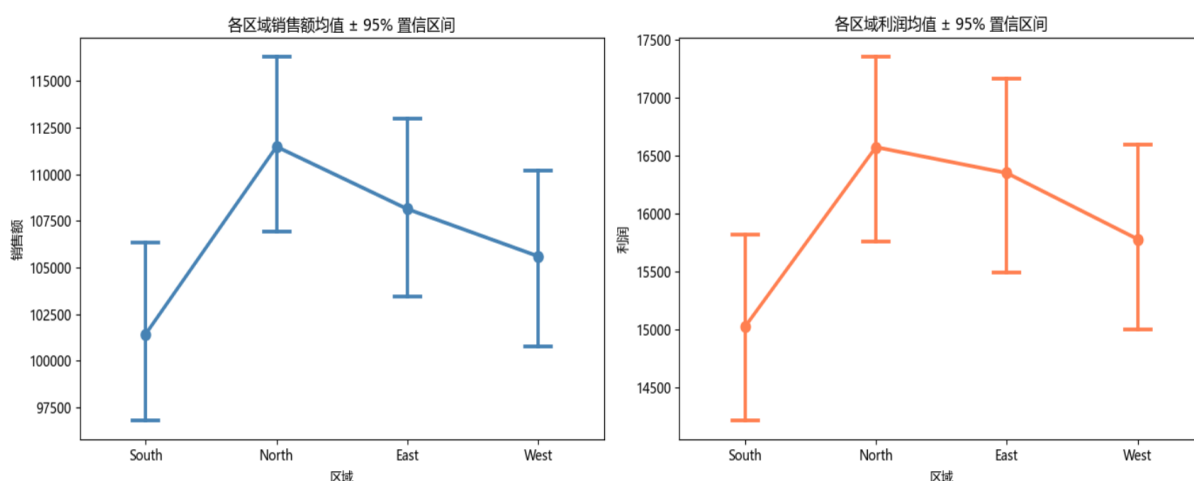
但是可以看出，利润的异常值相比销售额的异常值更多且分布更广，这说明有一部分的订单在利润这一维度上被放大了，平台在每个区域都存在规模不小的“暴利”订单，可能是高客单价与高利润率单品，可能是清仓售卖时的高利润遗留订单（成本极低但售价高昂，从一个高价降到没那么高的售价但依旧暴利），也可能是某些品类天然的高利润。

同时，不是每个高销售额的订单都是高利润的，有一些大额订单的利润较低，该订单可能因为过高的价格在销售额的箱线图里被统计为异常值，而在利润箱线图中不属于异常值；且存在着一批高利润订单，在总体销售额上正常，而利润尤其高，因而在利润箱线图中作为异常值被标注出来。

销售额是利润的基数，两者趋势一致说明利润主要由销售额驱动，而各区域之间不存在显著的利润差异，这能有效简化区域管理，也就是说，不需要针对各个区域设计不同的利润策略。

绘制各区域的销售额及利润的均值置信区间图，得到图 13。

图 13 置信区间图



由上图可以看出，各区域在销售额均值以及利润均值的表现趋势相似，再次验证了利润由销售额驱动。值得注意的是，南部与北部之间几乎没有重叠的置信区间，说明这两个区域之间的差异相较于其他区域之间的差异最显著。同时也可以注意到，各区域的区间宽度大致一致，说明其散布情况类似，而中心值存在差异。

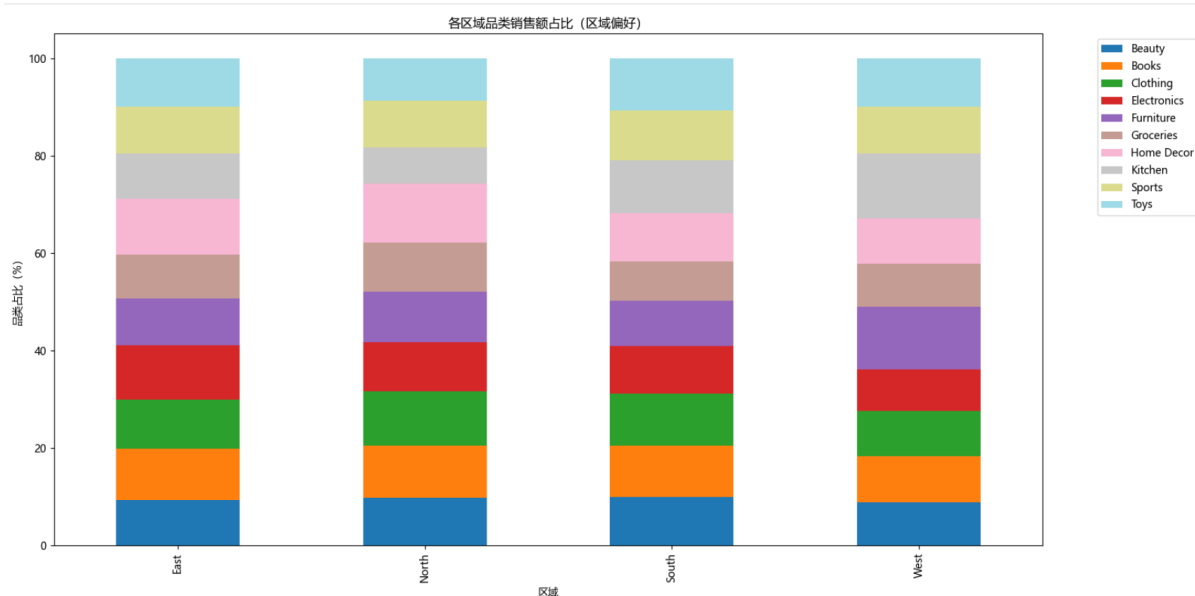
因此，可以认为各区域在销售额和利润的分布上存在差异，在某两个区域间的差距相对大一些，但整体差异并不是显著性的。北部区域对整个平台的贡献相对而言最大，在平台销售情况不那么尽如人意时可以考虑增加对北部区域的流量投放以及资源倾斜。而南部区域虽然贡献没有其他区域大，也不意味着需要针对该区域专门调整运营策略，在需要特别关照时可以结合该地区的品类偏好进行定向投流，从而增加销售额与利润。

4.1.2 各区域品类偏好分析

了解各区域间对商品品类的偏好趋势有助于调整营销策略，为商家及平台提供合理的决策建议。

对各区域中各品类销售额与总销售额的占比情况进行可视化，得到图 14。

图 14 各区域品类偏好



由上图可以看出，部分品类如 Beauty、Clothing、Sports、Books 的销售额在区域上的差异不明显。

分区域进行品类偏好分析：

East（东部）：该区域更偏好 Home Décor 及 Electronics 这两个品类，对 Groceries 相对不感兴趣；

North（北部）：该区域对 Home Decor 明显偏好，对 Kitchen 品类则兴致缺缺；

South（南部）：该区域对各品类的偏好差距不是很明显，但最不偏好的品类是 Groceries；

West（西部）：该区域对 Kitchen、Furniture 这两个品类情有独钟，对 Groceries 和 Electronics 这两个品类则兴趣不大。

由此得出，平台及商家在针对大区域进行投流引流时需重点关注各区域之间的品类偏好，向该区域明显偏好的品类增加流量，而对于没那么感兴趣的品类则不需要再额外分配资源以免引起用户反感。

4.2 各城市表现

4.2.1 城市销售额及利润率分析

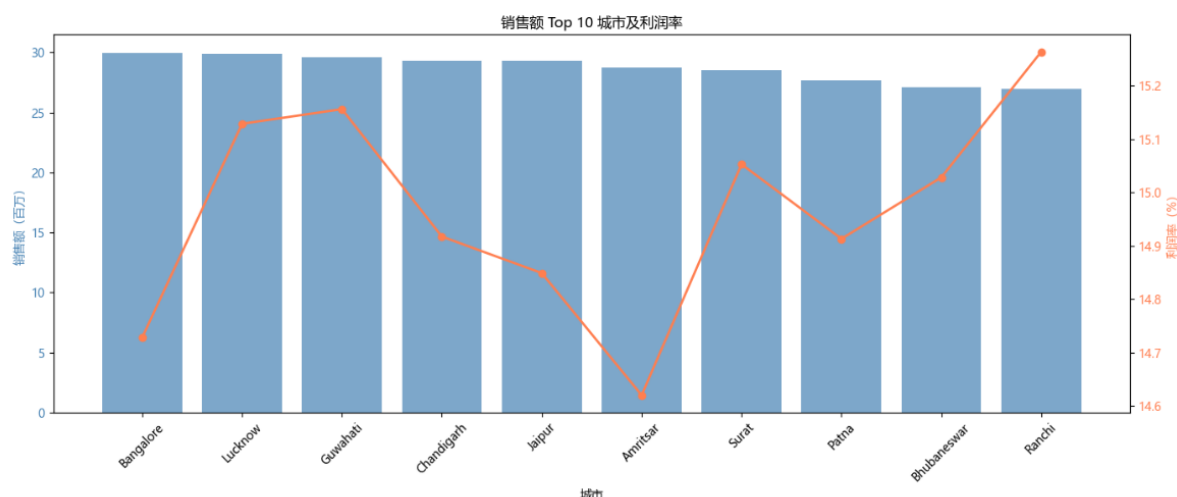
对各个区域的分析最终需要落实到城市分层上，哪些城市属于该平台的“头部城市”，规模如何，制定营销策略时若倾斜资源会得到多少效果，同时，哪些城市属于高销量高利润的双高城市，哪些城市存在“低销量高利润”或“高销量低利润”的情况，解决这些问题对平台健康发展，销售情况提升以及商家内部对各城市的营销策略调整都具有良好的积极的作用。

对数据集中各个城市进行汇总，首先取出其中销售额最高的 10 个城市作为代表进行分析，得到结果如下表 10 及图 15 所示。

表 10 销售额 TOP10 城市情况

City	Sales	Profit_Margin	Avg_Order_Value	Order ID
Bangalore	29,989,840.85	14.73	114,903.60	261
Lucknow	29,901,483.40	15.13	114,565.07	261
Guwahati	29,606,888.90	15.16	101,047.40	293
Chandigarh	29,331,057.35	14.92	106,271.95	276
Jaipur	29,319,402.90	14.85	112,334.88	261
Amritsar	28,746,184.20	14.62	115,912.03	248
Surat	28,532,934.65	15.05	109,321.59	261
Patna	27,702,815.20	14.91	107,375.25	258
Bhubaneswar	27,088,786.45	15.03	108,790.31	249
Ranchi	26,948,513.25	15.26	115,658.86	233

图 15 销售额 TOP10 城市及利润率分布



结合上表和图可以看出，头部城市确实存在，比如 Lucknow、Guwahati，同时具有高销售额以及高利润率，在资源分配时可以优先保障其仓储库存及物流，防止缺货导致的销量下滑。

同时也应注意，Bangalore 和 Amritsar 这两个城市出现了“高销量低利润率”的异常，需要对其进行深入分析，如折扣效应、品类结构、营销投放等方面是否存在缺陷，识别其利润流失的环节。

相应地，Ranchi 出现“低销量高利润率”的特征，在前十销量的城市中销量垫底但利润率最高，其运营模式值得进一步研究，识别并判断是哪些因素导致该结果以及该运营模式能否向其他城市进行推广复制。

值得注意的是，销售额排名前 5 的城市其销售额均超过 2900 万接近 3000 万，但之间的差距仍较明显，比如在第一二名的差距为 8.83 万的情况下第二三名之间以及第三四名之间分别差了 29.46 万和 27.58 万，同时第四五名之间又只差 1.17 万。由此可以看出，第一二名的城市属于第一梯队且竞争激烈，若延长时间线进行观察可能出现“轮流坐庄”的情况；由于断层的销售额差距，第三第四名无法归为同一个梯队，需要对第四名的 Chandigarh 进行原因排查，是什么导致它不能进入第二梯队，而是和第五名处于同一梯队。该断层情况在前 10 名的城市中已经出现明显特征，说明各个梯队以及各个城市的营销策略必须有所侧重，例如第一梯队需要保持竞争力，在大的方面不需要进行调整而是对精细化的小方面进行微调；断层处需重点分析究竟是市场规模、地理位置还是运营效率或是其他因素对其产生了影响。

对数据集中的所有城市进行销售额均值及利润均值的分布进行可视化，将销售额前十的城市用红色进行标注，得到图 16。

图 16 各城市销售额均值、利润均值散点图

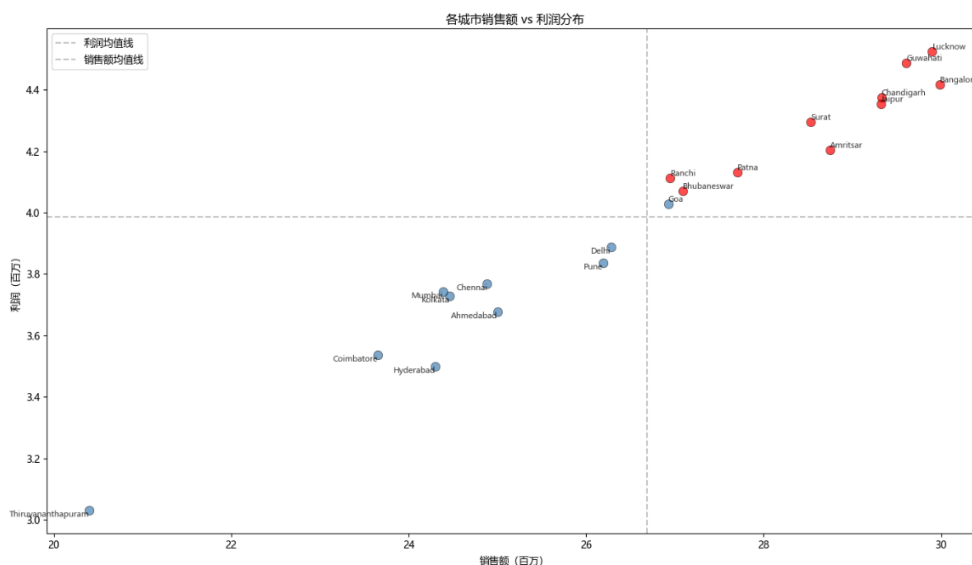
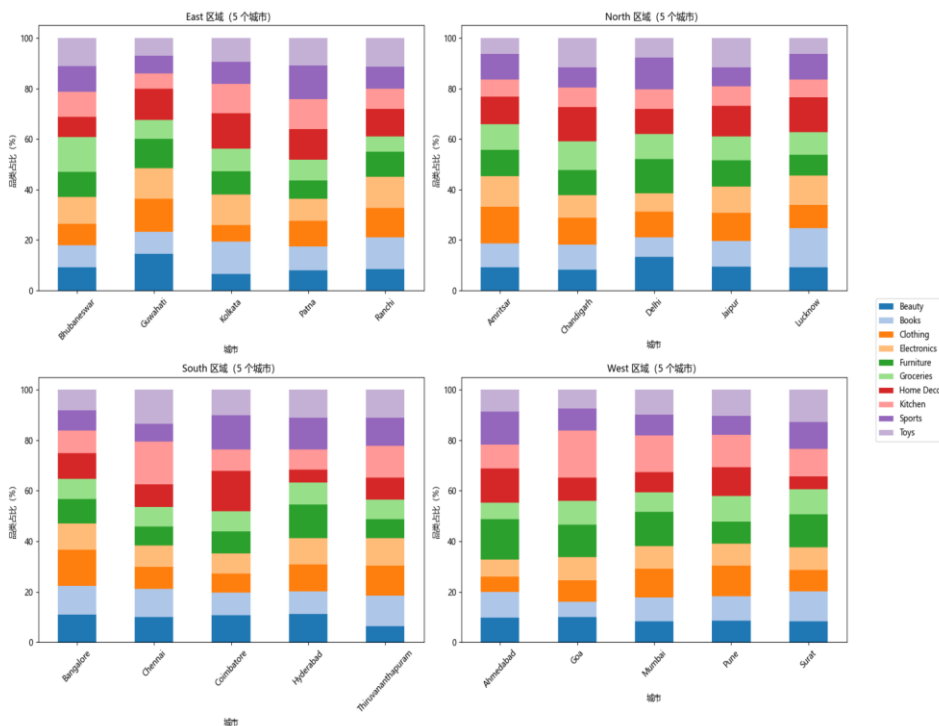


图 16 进一步印证了结论，城市之间存在明显的梯队差异，而最末的城市则是断崖式的低，且各梯队内部竞争激烈，需要对出现明显断层的城市进行运营策略分析。

4.2.2 按区域的城市偏好分析

首先对各区域包含的城市进行汇总，可以得到东南西北四个区域内各有 5 个城市，共计 20 个城市，分布较为均匀。再按照区域对各城市的品类偏好进行可视化，结果如图 17 所示。

图 17 按区域的城市偏好可视化



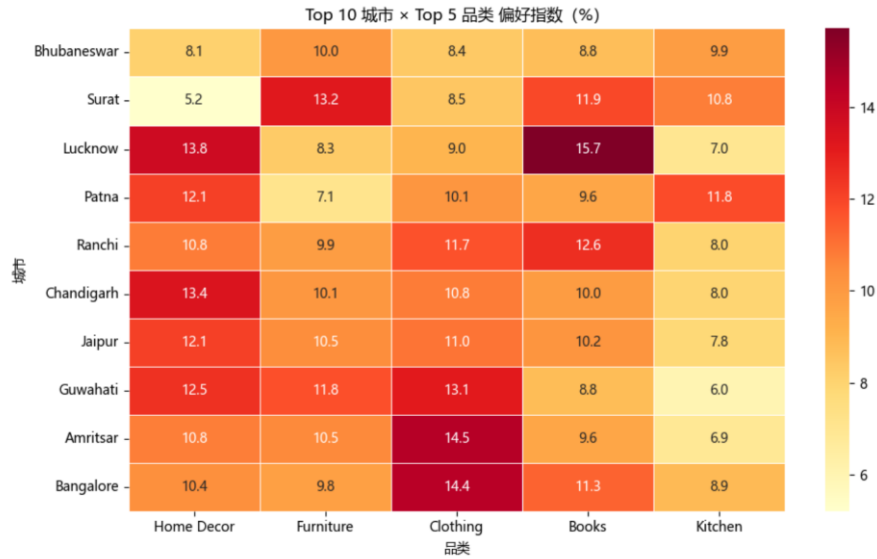
由上图可知，区域内各城市的品类偏好存在明显差异，且城市与城市之间的偏好也没有雷同。说明针对城市的品类偏好分析具有其研究价值，对区域内部各城市的营销策略调整具有指导作用。

以 South（南部）区域为例：该区域中，Toys、Kitchen 品类的营销策略应重点关注 Chennai，Sports、Home Décor 品类则需要关注 Coimbatore，Groceries 及 Electronics 品类在该区域则应当均匀配置资源，Furniture 需要将资源向 Hyderabad 倾斜一部分，Books 在 Thiruvananthapuram 需要增加投流，而 Beauty 则需要减少在该城市的资源分配而更关注其他城市，Clothing 则需要对 Bangalore 投入更多的资源。

4.2.3 城市品类偏好分析

对销售额前十的城市以及销售额前五的品类进行交叉分析，可视化得到图 18。

图 18 城市品类交叉表



从组合层面进行分析，由上图可得，Lucknow×Books、Amritsar×Clothing 及 Bangalore×Clothing、Lucknow×Home Decor 及 Chandigarh×Home Decor 为热力值排名前五的五对组合。即在 Lucknow 区域加大 Books、Home Decor 品类的营销投入以及库存保障，Clothing 品类的品牌方或商家应在 Amritsar 与 Bangalore 投入更多的营销及仓储准备，针对 Chandigarh 的 Home Decor 品类营销策略需要增加投入及各项保障。

从城市品类集中度进行分析，不难发现 Bhubaneswar 整体对这五个品类没有特别的偏好，即该城市具有相对均衡稳定的品类结构，各个品类的需求量几乎持平，若某个品类出现波动也不会对该城市的整体销售额造成大的影响。Chandigarh、Jaipur 的品类结构也相对而言比较稳定，这两个城市在 Kitchen 品类的偏好相较于其他品类偏低，因而资源可以少往该品类分配，往其他品类流动。

同时，Surat、Lucknow、Patna、Ranchi 等城市对某几个品类的商品高度依赖，需要重点关注这几个品类的仓储库存和物流情况，若重点品类出现波动则该城市的销售额会受到较大冲击。

从品类区域特征进行分析，Clothing、Books 品类在各个城市均有不错的表现，属于刚需品类，而 Home Décor、Furniture 品类仅在两个城市表现略差，也可以作为刚需品类，在此情形下刚需品类可以认为是热销品类，可以采取统一采购的方式降低成本。

Kitchen 品类仅在 Patna 等三个城市具有良好表现，可以针对重点区域进行资源倾斜，同时调研该品类在其他城市的情况，判断是市场规模还是营销策略的问题。

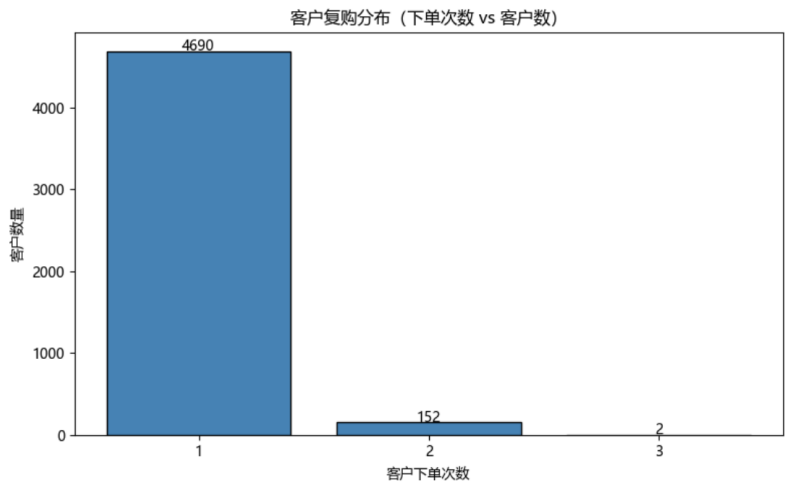
五、用户行为分析

5.1 客户复购分析

随着大数据时代的到来，电商行业的发展迎来了全新的机遇和挑战，基于电商平台的海量用户行为数据，如何深入挖掘并精准预测消费者的复购行为，对电商平台提高客户忠诚度和运营效果有着重要意义。^[3]

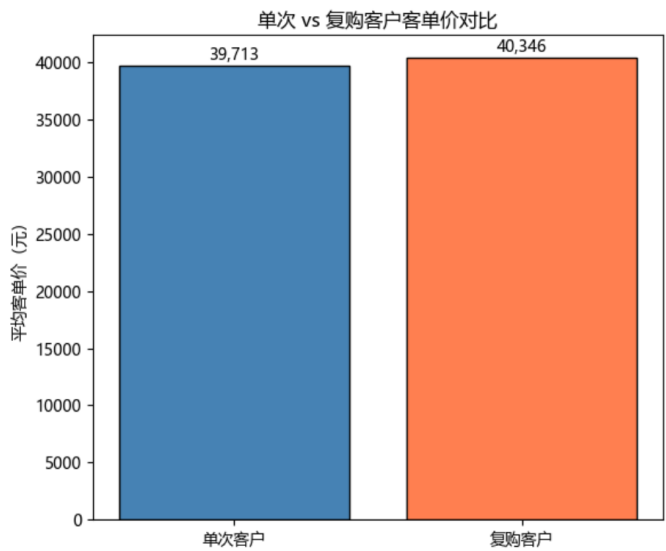
首先对数据集中的用户复购情况进行汇总。如图 19 所示。

图 19 用户复购情况汇总



可以得出，单次客户为 4690 人，复购客户仅有 154 人，复购率为 3.2%，5000 条数据记录中有 4844 名用户。

图 20 客户客单价对比



对单次客户及复购客户的平均客单价进行对比得到上图，单次客户的客单价为 39713 元，复购客户的客单价则为 40346 元，仅差 633 元，相对差只有 1.6%。这意味着该平台的复购客户并没有比单次客户花费更多的钱，复购客户也仅仅是回平台再买一次，但每次的消费水平和单次客户没有什么特别显著的区别，复购并不是该平台实现营收增长的杠杆，在短期内平台应该将重点放在新顾客的吸引，而不是引导客户

复购；中期可以针对中高客单价客户进行售后回访或是关联推荐，验证高价值客户复购可能性更大的猜测；长期来看需要平台巩固客流，但还是需要平台尝试提高客单价。

同时也说明，该平台的客户价值差异并不在于客户重复购买的次数，不在于“回头客”的数量，而在于客户下了什么订单和买了多贵的商品。为验证该推测的准确性，对客户价值进行分层分析。

5.2 客户价值分层

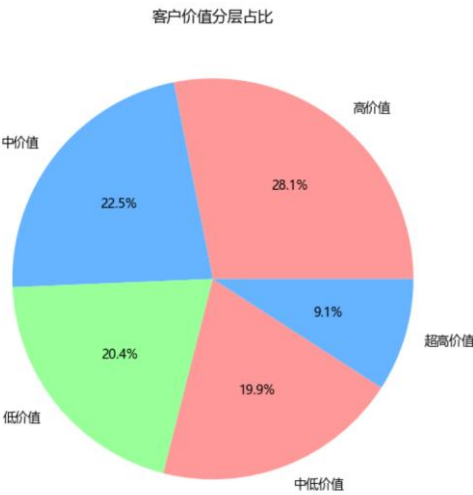
首先对数据集中各个用户的消费金额进行统计分析，得到表 11。

表 11 用户消费情况统计

count	4,844.00
mean	110,170.53
std	88,632.70
min	264.10
20%	32,645.12
40%	63,785.70
50%	85,848.50
60%	112,827.20
80%	183,411.12
90%	242,705.66
95%	289,507.25
99%	357,133.60
max	650,151.90

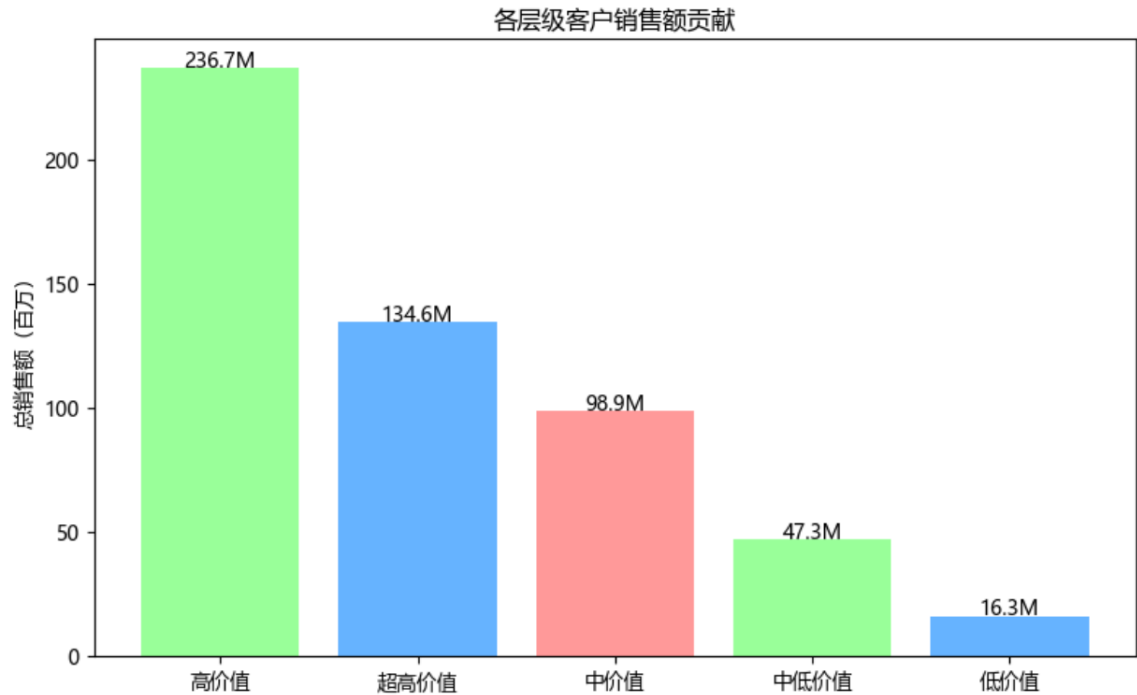
由表 11 可以得到，前 20%的用户消费金额为 32645.12，前 40%的用户消费金额为 63485.7，前 60%的用户消费金额为 112827.2，前 90%的用户消费金额为 242705.66。根据表 11 对客户以金额进行分层，分位数定为 32000、64000、120000 及 250000，分类为“低价值”、“中低价值”、“中价值”、“高价值”及“极高价值”。可视化各层用户占比得到图 21。

图 21 客户价值分层占比



因分层时采用了固定分位数的分类策略，在前三个层级中各层用户占比基本为20%，在高价值和极高价值的客户中存在差异，头部客户占9.1%。对各层级用户对销售额的贡献进行可视化，得到图22。

图 22 各层级客户销售额贡献



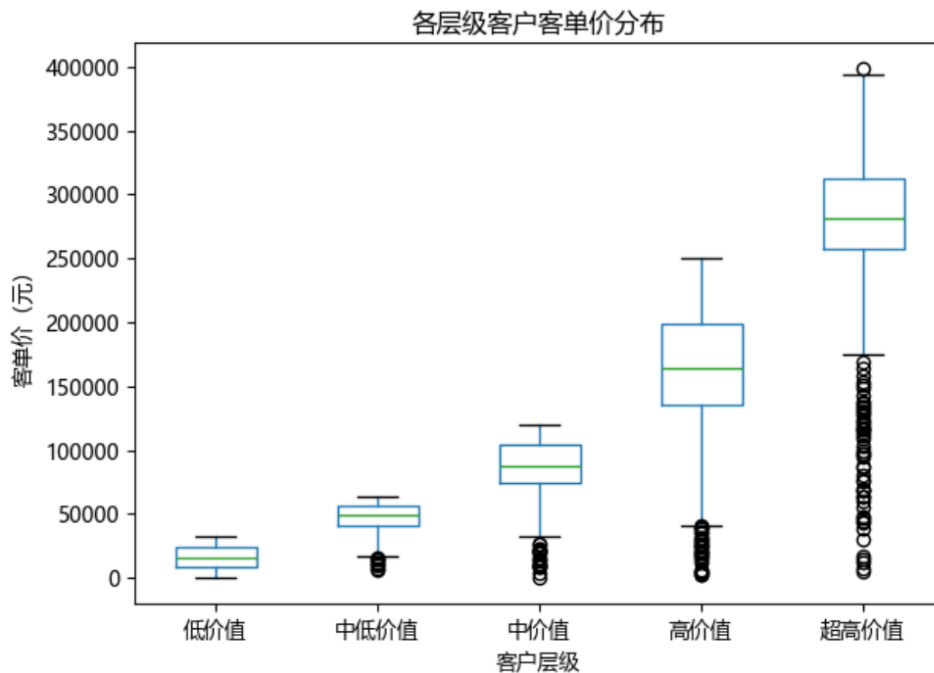
可以发现，占比28.1%的高价值用户对销售额的贡献最大且呈现断层式的贡献，比超高价值用户及中价值用户的销售额总和还多320万；而中低价值与低价值客户的贡献之和只占中价值客户贡献的35.69%。

这说明在该平台上，销售额并不靠头部极少数（9.1%）的客户拉动，也并不依赖中层的大多数客户，而是依赖紧挨着头部的那部分高价值客户拉动，这批客户才是真正的消费主力，应当重点维护，防止其流失侵蚀平台销售额。

相应地，中层的大部分用户并未起到承接作用，“过渡层”的设置反而并未起到其过渡作用，客户要么直接买到高价值，要么一直停留在中价值以下的位置，缺乏有效的承托机制让客户实现层级的跃迁。

同时，占比40.3%的中价值以下的客户对销售额的贡献仅占总额的11.91%，接近半数的客户几乎不贡献销售额，说明该平台目前依赖少数核心客户支持，更说明了需要对核心用户进行维护，且不要对下沉用户过度投入，不要为了扩大市场盲目扩展中低价值客户。

图 23 各层级客户客单价分布



对各层级客户客单价分布进行可视化，得到图 23。

不难看出，随着层级的提升，各层级客户的客单价中位数呈现上升趋势，且箱体之间并不存在明显的重叠现象，各层级客户的边界清晰，说明该分层方法合理且有效，阈值设定合理。以高价值与超高价值为例：高价值层中最顶尖的客户刚好能够到超高价值的门槛（即超高价值的下四分位点），而超高价值的客单价最低的客户大约等于高价值层的中等水平。

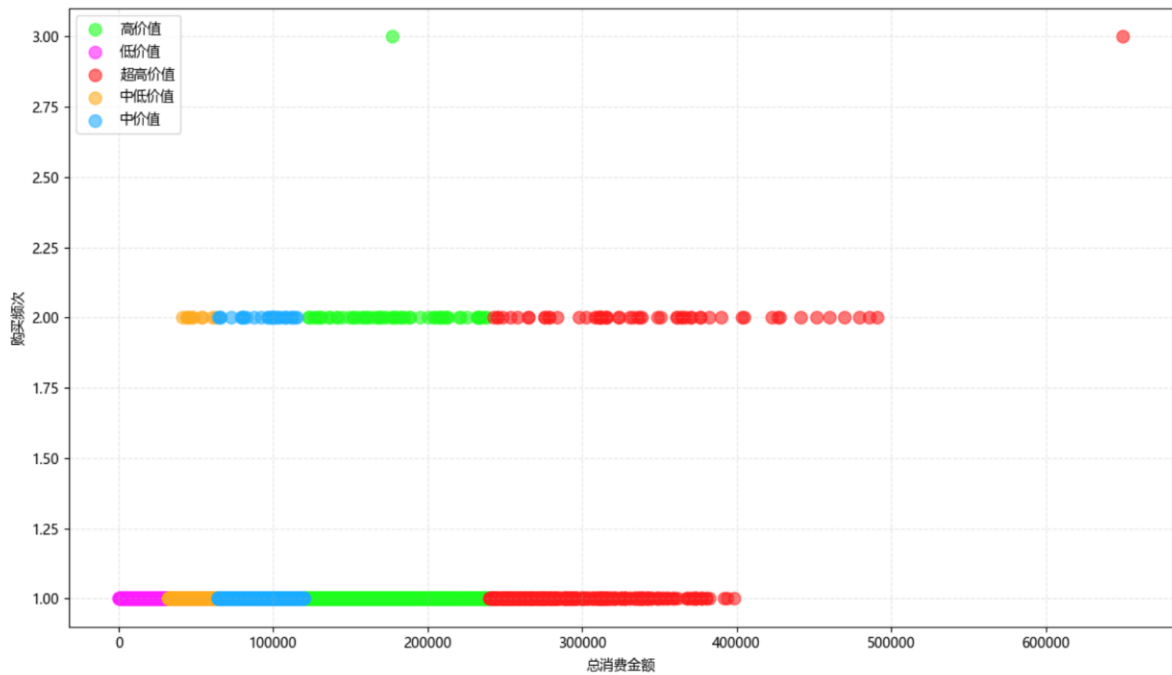
从箱体高度可以得到，低价值客户及中低价值客户呈现低离散的特征，说明该层级内客户行为具有高度的一致性，在运营时不需要对层级内的客户分类进行维护；而高价值及超高价值的客户呈现高离散的特征，说明层级内部存在着明显的差异，可以进一步细分。且高价值层的箱体显著变长，须线长度也最大，这意味着在所有层级中，高价值层是客户行为最不统一的群体，同样被归类为高价值，客单价之间存在着明显的差异。

值得注意的是，从中低价值层级开始，须线下方开始出现异常值，且随着层级上升而逐渐增多，超高价值层级具有最多的下方异常值，且在上方具有唯一异常值。这是一个明显的预警信号。

低价值层级不存在异常值且箱体须线都很紧凑，该层级满足客单价低且总消费低的特征，表里如一，并不需要格外关注。**中低价值**层级开始出现下方异常值，说明存在着总消费额符合分层水平但客单价极低的用户，结合该平台 3.2%的复购率可以推测这部分被归为异常的客户属于用购买次数增加总消费额换来的层级提升，且在**超高价值**这一层级尤其集中，说明依靠消费总额对客户进行分层实际上掩盖了很多关键信息，一锤子买卖的大单客户和忠诚度较高的复购客户被混在一起，运营策略无法精准匹配。

引入购买频次作为第二个分层纬度，绘制客户总消费金额与购买频次的散点图(图 24)以及不同购买频次客户的客单价分布(图 25)。

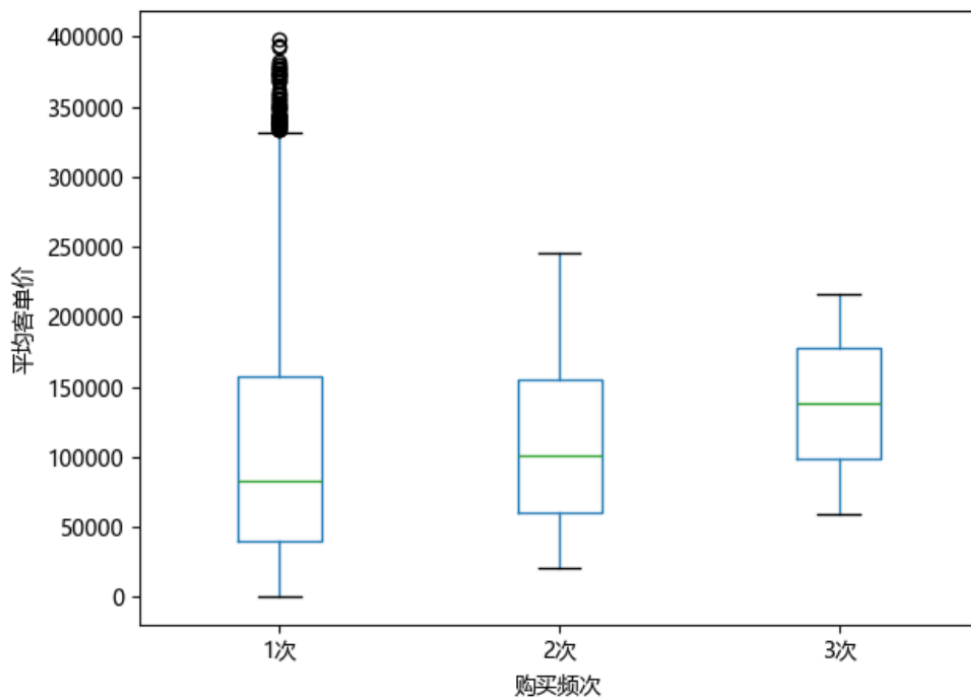
图 24 消费总额与购买频次散点图



针对购买总额进行顾客分层，同时结合购买频次进行观察，可以发现，这样的分层结构展示了清晰的“复购行为梯度”，低价值层级 100% 一次性购买，中低价值层级开始出现复购行为，中价值层级复购行为较前一层级更明显，高价值层级中复购客户占主力，超高价值层级中复购行为与超大单共存。复购行为的渗透率随着层级上升逐渐提高。

不难看出，低价值层级客户是天然的一次性小额客户，制定运营策略时无需过多纠结；中低价值到中价值的层级之间是复购行为渗透的关键区间，需要对客户复购品类及时间进行进一步的深度调研，了解什么因素影响了复购行为；高价值客户中复购客户与一次性大单客户平分秋色，但两者需要完全不同的运营策略；而超高价值层的复购比例最高。说明持续的复购行为是通往更高价值的手段。

图 25 不同购买频次客户的平均客单价分布



由图 25 可以简单得出，相较于消费总额而言，购买频次是一个更值得关注的价值信号。随着购买次数的增加，客单价的底线在逐渐上升，从 222 元提升至接近 60000 元，中位数从 80000 元左右提升 140000 元，上涨了约 75%，且客单价分布从极度离散收窄到了相对集中（1 次的 IQR 约为 110000, 2 次的 IQR 约为 95000, 3 次的 IQR 约为 70000），异常值也随之消失。

不难得出，对该平台而言，复购并不是同一批客户买着同样的商品，而是筛选得到了具有更高的支付能力的客户。购买次数达到 3 次的客户，其最低客单价远超过 1 次购买客户的 Q1，接近 2 次购买客户的 Q1，这意味着这批客户整体位于一个更高的消费层级。

针对该平台 3.2% 的复购率，可以发现单次购买客户（4690 人）包罗万象成分复杂，同时具有五个层次的消费群体且客单价跨度极大，运营难度也最高，大量的客户单次支付金额集中在 40000 到 150000 之间，多数人的客单价并不高，但少部分大单（最高高到 400000）把整体的上限拉得很高，而且有不少一单就把自己送进“高价值”或“超高价值”层的客户，导致这批客户的分散程度极高，无法使用统一的运营策略覆盖，运营难度也最大。

而二次购买的客户（152 人）未出现异常值，最低约为 25000（远高于 1 次），最高约为 250000（低于 1 次的异常值以及大部分大单），且集中程度比单次购买客户好很多。不难看出二次购买筛掉了那群低价值层级的客户，客单件低于 25000 的人不会再复购，可能这批客户的需求被一次满足，或者是购物体验不够好不考虑回购；而一次性买到 400000 以上的超级大客户同样也没有复购，可能是冲动消费或者是一次性需求。二次购买筛去了低净值客户，同时也压住了极端客户，但其用户行为具有可预测的规律特征（客单价未出现异常值）。

在数据集中 3 次购买的客户只有两人，但具有极其重要的研究意义。箱线图的箱体居中，是所有组中唯一不偏态的，说明这两位客户的购买行为最正常也最稳定，具有最强的可预测性，但仍可以看出，就算是只有两人，其差异也十分巨大，一个客单

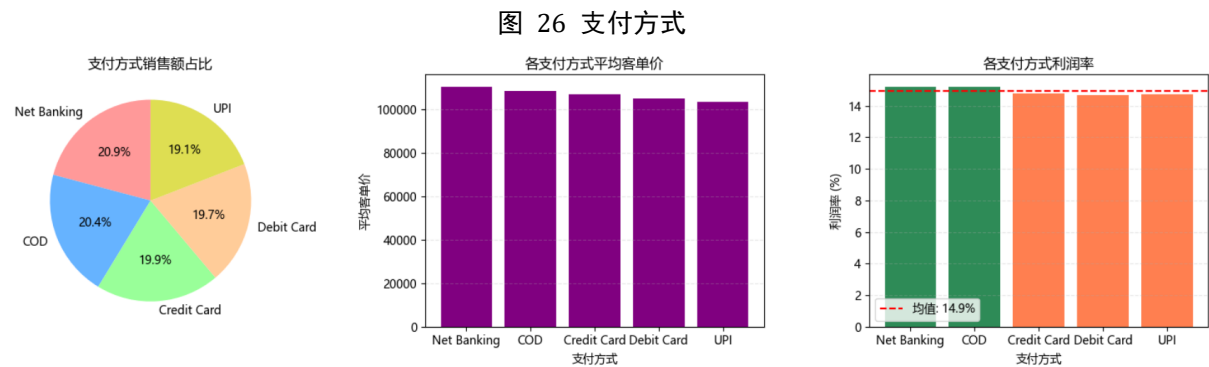
价 6 万左右而另一个客单价 20 多万，即使都是“忠实客户”也存在着 3.5 倍以上的消费力差距。结合图 24 可以看出，一位是高价值用户，另一位是超高价值用户，其本身层级不同，运营策略也自然存在差异。

5.3 支付方式分析

对各订单使用的支付方式进行汇总，计算其销售额、利润、利润率以及平均单价得到表 12，对各支付方式在总销售额的比例、平均客单价及利润率进行可视化，得到图 26。

表 12 支付方式情况分析

Payment Mode	Sales	Profit	Order ID	Profit Margin	Avg_Order Value
Net Banking	111,465,516.05	16,964,119.74	1010	15.22	110,361.90
COD	108,881,396.25	16,575,845.53	1005	15.22	108,339.70
Credit Card	106,027,144.90	15,658,639.35	994	14.77	106,667.15
Debit Card	105,346,381.65	15,479,323.07	1003	14.69	105,031.29
UPI	101,945,585.50	15,030,807.22	988	14.74	103,183.79



不难看出，各支付方式在销售额、利润、利润率以及平均客单价上的表现没有显著的差异，这个维度上来说没有哪个支付方式占据绝对主导地位，客户在选择支付方式时也没有特定的偏好，暂时不作为运营优化、策略调整的重点方向。

六、结论与建议

6.1 结论

1. 销售趋势：平稳波动，季节性显著但存在结构性隐患

该电商平台整体的销售额与利润呈平稳波动，具有明显季节性（5月、7月为天然旺季，11-12月为促销活动带来的营销旺季，1月-4月为销售的淡季，尤其是1月，比全年平均低了8.9%）。使用时间序列模型对销售额进行预测，ARIMA模型预测未来半年的月销售额稳定在2250万元左右。然而，2025年9月的实际销售额（1813万）比模型预测的2247万低了19.3%，偏差值达到23.9%，这一偏差在ETS模型中更为显著，达到了27.7%，同时2025年5月的真实销售额（2601万）比模型预测的2249万高了15.7%，在ETS模型中同样有所体现，说明这两个月可能发生了模型无法捕捉到的业务变化，建议运营团队可以对这两个月进行重点复盘排查原因，需要结合业务日志和竞品动态等因素综合复盘，否则若下一轮同类事件发生时，模型预测将再次失去意义。

2. 品类结构：高度分散，Furniture为利润核心

平台的HHI指数为0.1003，前三大品类累计占比仅31.7%，属高度分散的综合性平台，无单一品类依赖风险。帕累托曲线呈现单调增长形态，而非传统L型或S型，进一步验证了品类贡献的均衡性。。Furniture品类销售额排名第二、利润率排名第一，为“利润明星”；与之相对，Groceries销售额垫底、利润率倒数，Books利润率倒数第三，属薄利品类，适合引流但不宜作为利润来源——Groceries更多是为满足用户一站式购物需求而保留的防守型品类。不应投入过多营销预算。品类折扣敏感度分析显示，多数品类折扣对销量拉动几乎无效（相关系数接近0），其中Books品类相关系数甚至为-0.13，出现“打折越多卖得越少”的负敏感现象，这意味着平台目前的折扣更多是“利润转移”而非“增量创造”，建议平台减少无差别打折。

3. 地域表现：差异存在但不悬殊，城市梯队分化明显

各区域销售额与利润存在统计上的显著差异，ANOVA p值分别为0.026与0.046，但p值均接近显著性临界值，差异幅度不大——北方贡献最高（1.44亿），南方相对偏低（1.23亿），极差约2,000万。箱线图显示各区域销售额分布高度重叠，均值差异在可视化层面并不突出，统计显著性主要来自大样本量放大了微小差异。城市层级分析识别出明显梯队分化，这种“竞争胶着—突然断层—又恢复胶着”的梯队模式，说明城市间并非均匀递减，而是形成了明显的层级壁垒，需差异化配置资源：第一梯队维持优势并引入良性竞争，第二梯队集中资源扶持1-2个城市突破断层。而区域的品类偏好存在着明显差异，East偏好Home Décor与Electronics，North偏好Home Décor，West偏好Kitchen与Furniture，South品类结构最均衡。

4. 用户行为：拉新驱动，高价值客户断层式领先

平台复购率仅3.2%，单次客户占比96.8%，属典型的拉新驱动型业务。客户价值分层显示，28.1%的高价值客户贡献呈断层式领先——比超高价值与中价值客户总和还多320万元，而40.3%的中低及低价值客户仅贡献11.9%的销售额。单次客户平均客单价为39,713元，复购客户平均客单价为40,346元，两者仅差633元（1.6%），说明复购客户并未比单次客户花更多钱。但引入购买频次进行二维交叉分析后发现：

购买 3 次的客户客单价中位数较单次客户提升约 75%，且客单价分布从极度离散收窄至相对集中（IQR 从 11 万降至 7 万），说明复购行为实质上是筛选高支付能力客户的有效信号，而非单纯的购买次数积累。

6.2 建议

1. 促销策略：停止无差别打折，转向差异化定价

打折对多数品类销量拉动几乎无效，建议将原本用于折扣的预算转向广告投放与引流活动。对不同品类制定差异化促销策略：Furniture、Electronics 等高利润品类以“满减”替代“直接打折”；Books、Groceries 等薄利品类通过捆绑销售或会员价方式促销，避免直接降价侵蚀利润。

2. 用户运营：分级维护，分层施策

针对高价值客户（占比 28.1%）建立专属维护机制，提供优先发货、专属客服、新品优先购买权等权益，防止核心用户流失。将高价值客户进一步拆分为“单次爆发型”（单次消费极高但无复购，集中在超高价值层）与“频次堆叠型”（多次购买累计达标，随层级上升复购渗透率递增）两类，前者通过售后回访和关联推荐引导二次转化，后者纳入会员体系给予持续激励。针对中低及低价值客户（贡献仅占 11.9%），通过自动化营销触达（邮件、推送），不投入人工运营，避免资源浪费。

对于复购率极低的问题，建议聚焦推动客户完成第 2 次购买：客单价低于 2.5 万的单次客户自然流失，无需过度干预；重点筛选单次消费在 2.5 万以上且购买时间在 30 天内的客户，进行定向复购触达。

3. 区域与城市策略：按梯队配置资源

第一梯队城市（Bangalore、Lucknow）维持现状，通过精细化运营争夺榜首。第二梯队城市（Guwahati、Chandigarh、Jaipur）集中资源扶持 1-2 个城市突破断层，重点分析断层产生的原因（市场规模、运营效率还是供应链问题）。针对“高销量低利润率”城市（Bangalore 利润率 14.73%、Amritsar 利润率 14.62%，均低于均值 14.94%），排查折扣力度、品类结构和物流成本，识别利润流失环节；针对“低销量高利润率”城市（Ranchi 利润率 15.26%），研究其高利润原因并评估向其他城市复制的可行性。

4. 季节性运营：提前备货，平滑波谷

在 5 月、7 月、11 月旺季前 4-6 周提前备货，保障库存和物流运力。1 月、3 月、9 月淡季主动策划促销活动，如“换季清仓”与“新品首发”结合，对冲自然下跌。将 ARIMA 预测建立偏差根因登记机制，积累业务事件标签（促销/缺货/竞品动作），未来引入外部变量构建 SARIMAX 模型接入月度运营看板，持续跟踪预测偏差，当偏差超过 15% 时自动触发预警。

6.3 研究局限与展望

数据集时间跨度为 24 个月，对于 ARIMA 等需要多周期估计的季节性模型而言训练数据不足（训练集仅 18 个月，不足 2 个完整季节周期），导致季节性参数无法稳定估计。未来可积累更多月度数据（建议 48 个月以上），重新评估季节性模型的适用性。此外，数据集缺少退货、退款、客户浏览行为、营销投放等字段，限制了分析的深度——如有退货数据可分析净销售额与实际利润，有营销数据可评估 ROI，有浏览行为

数据可构建更精准的转化漏斗。缺少获客渠道等字段，无法对各个渠道的客户质量进行区分。

本研究采用 ARIMA(1, 1, 1) 作为预测模型，其预测值趋于平稳，围绕 2,250 万元小幅波动，对突发事件的响应能力有限。未来可引入外部变量，如节假日虚拟变量、营销活动日历等，构建 SARIMAX 或 Prophet 模型，提升预测精度。客户价值分层采用基于消费金额的静态分层，未考虑客户生命周期的动态变化，后续可引入迁移矩阵追踪客户在各层级间的跃迁路径

复购分析受限于复购客户样本量较小（仅 154 人，占客户总数 3.2%），结论的统计稳健性有限，复购行为的因果机制尚未厘清，尤其是 3 次购买的客户仅有 2 人，箱线图解读更多是定性方向而非定量结论，需要更长观察期积累样本。2025 年 5 月和 9 月出现的预测偏差提示存在模型未能捕捉的外部因素，后续需结合业务日志和竞品动态进行根因定位。品类折扣敏感度分析中多数品类相关系数接近 0，但不排除折扣与销量之间存在非线性关系（如折扣超过某一阈值后才有效），后续可引入分段回归或 U 检验进行验证。同时未对客户初次下单时间进行补充，数据集仅包含 2023 年 10 月 4 日开始的数据，无法确认其中的顾客在之前的时间内是否有购买行为，更无法判断在该数据集中的单次客户是“新客”还是“流失老客”。

最重要的是，平台的增长引擎尚未定性。本报告揭示了平台“拉新驱动”与“复购率 3.2%”之间的结构性矛盾，但受限于数据维度，未能回答一个根本问题：该平台究竟靠“持续获取大额一次性客户”盈利，还是应转向“客户生命周期价值驱动”模式？若为前者，当前分析框架中缺少获客成本（CAC）数据，无法判断拉新驱动模式是否可持续、是否真正盈利；若为后者，则 3.2% 的复购率是致命短板，需要从商品结构、服务体验、用户触达等层面进行系统性重构。后续研究建议将获客成本纳入分析，并结合品类天然复购周期，明确平台商业模式的本质定位。

七、参考文献

- [1] 葛娜. 基于 ARIMA 时间序列模型的销售量预测分析[J]. 北京联合大学学报, 2018 年 10 月
- [2] 卫志民编. 微观经济学. 高等教育出版社. 2011. 07
- [3] 李盼, 杨继宇. 基于集成学习的电商消费者复购行为预测及可解释性分析[J]. 电子商务评论, 2025, 14(7): 2525-2534. DOI: 10.12677/ecl.2025.1472463

八、附录

程序：

```
#导入库
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#直接显示图表
%matplotlib inline
#数据读取
df = pd.read_csv('Ecommerce_Sales_Data_2024_2025.csv')
df.head()
df.info()
#数据处理
#判断是否存在缺失数据
print(df.isnull().sum())
#判断是否存在重复数据
print(df.duplicated().sum())
#数据概览
print(df.describe())
#定位“异常数据”判断该数据是否需要剔除
print('客单价最低的记录：')
print(df[df['Unit Price']==df['Unit Price'].min()])
print('\n'+ '-' *80)
print('利润最低的记录：')
print(df[df['Profit']==df['Profit'].min()])
#将下单日期转换为日期格式
df['Order Date'] = pd.to_datetime(df['Order Date'])
#提取年份及月份
df['YearMonth'] = df['Order Date'].dt.to_period('M')
df['Year'] = df['Order Date'].dt.year
df['Month'] = df['Order Date'].dt.month
#查看数据基本情况
print(f"总行数：{len(df)}")
print(f"日期范围：{df['Order Date'].min()}至{df['Order Date'].max()}")
print("\n各品类销售额占比情况：")
print(df.groupby('Category')['Sales'].sum().sort_values(ascending=False))
#相关性分析
corr_matrix = df[['Sales', 'Profit', 'Discount', 'Quantity', 'Unit Price']].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fnt='.2f', linewidths=0.5)
plt.title('Correlation Matrix')
```

```

plt.show()
#销售趋势分析
#使用中文字体
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False

#1、整体销售趋势分析
monthly_data = (
    df.groupby(df['Order Date'].dt.to_period('M'))[['Sales', 'Profit']]
        .sum()
        .reset_index()
)
monthly_data['Order Date'] = monthly_data['Order Date'].dt.to_timestamp()

fig, ax1 = plt.subplots(figsize=(14, 5))
ax2 = ax1.twinx()
ax1.plot(monthly_data['Order Date'], monthly_data['Sales'], color='steelblue', marker='o', linewidth=2, label='
销售额')
ax2.plot(monthly_data['Order Date'], monthly_data['Profit'], color='coral', marker='s', linewidth=2, label='
利润')

ax1.set_xlabel('月份')
ax1.set_ylabel('销售额', color='steelblue')
ax2.set_ylabel('利润', color='coral')
ax1.legend(loc='upper left')
ax2.legend(loc='upper right')
plt.title('月度销售与利润整体趋势 (2023-10 ~ 2025-10)')
plt.grid(True, linestyle='--', alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

#2. 季节性销售趋势分析
monthly_pattern = df.groupby(df['Order Date'].dt.month)['Sales'].mean().reset_index()
monthly_pattern.columns = ['Month', 'Avg_Sales']

# 相对于全年平均的月度指数
avg_all = monthly_pattern['Avg_Sales'].mean()
monthly_pattern['Index'] = (monthly_pattern['Avg_Sales'] / avg_all - 1) * 100

# 柱状图绘制
colors = ['red' if x < 0 else 'steelblue' for x in monthly_pattern['Index']]
plt.figure(figsize=(12, 5))

```



```

plt.bar(monthly_pattern['Month'], monthly_pattern['Index'], color=colors, edgecolor='black')
plt.axhline(y=0, color='black', linestyle='-', linewidth=0.8)
plt.xlabel('月份')
plt.ylabel('偏离均值百分比 (%)')
plt.title('月度销售额季节性指数 (正 = 高于全年平均, 负 = 低于全年平均)')
plt.xticks(range(1, 13), ['1月', '2月', '3月', '4月', '5月', '6月', '7月', '8月', '9月', '10月', '11月', '12月'])
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()

# 结论
peak_month = monthly_pattern.loc[monthly_pattern['Index'].idxmax()]
trough_month = monthly_pattern.loc[monthly_pattern['Index'].idxmin()]
print(f"销售高峰期: {int(peak_month['Month'])}月 (比平均高 {peak_month['Index']:.1f}%)")
print(f"销售低谷期: {int(trough_month['Month'])}月 (比平均低 {trough_month['Index']:.1f}%)")

#时间序列预测销售趋势
#导入库
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error

#数据处理
#检查月份是否连续
monthly_sales = df.groupby('YearMonth')['Sales'].sum()
monthly_sales.index = monthly_sales.index.astype(str)
monthly_sales = df_ts.groupby(df_ts['Order Date'].dt.to_period('M'))[['Sales', 'Profit']].sum().reset_index()
monthly_sales['Order Date'] = monthly_sales['Order Date'].dt.to_timestamp()

print(monthly_sales.tail(10))
print(f"总月份数: {len(monthly_sales)}")
# 画原始序列
plt.figure(figsize=(14, 4))
plt.plot(monthly_sales['Order Date'], monthly_sales['Sales'], marker='o', linewidth=2)
plt.title('Origin Sequence (2023-10 ~ 2025-9)')
plt.xlabel('MONTH')
plt.ylabel('Sales')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

# 用前 18 个月训练, 后 6 个月测试
train = monthly_sales.iloc[:-6]

```

```

test = monthly_sales.iloc[-6:]

print(f"训练集: {train['Order Date'].min()} ~ {train['Order Date'].max()} ({len(train)}个月)")
print(f"测试集: {test['Order Date'].min()} ~ {test['Order Date'].max()} ({len(test)}个月)")

#ETS 模型
ets_model = ExponentialSmoothing(
    train['Sales'],
    trend='add',
    seasonal='add',
    seasonal_periods=6
).fit(use_brute=False)

ets_forecast = ets_model.forecast(6)
ets_mae = mean_absolute_error(test['Sales'], ets_forecast)
ets_rmse = np.sqrt(mean_squared_error(test['Sales'], ets_forecast))
print(f"ETS MAE: {ets_mae:,.2f} RMSE: {ets_rmse:,.2f}")
print(f"预测值: {ets_forecast.round(0).tolist()}")

plt.figure(figsize=(14, 5))

# 测试集真实值
plt.plot(test['Order Date'], test['Sales'],
         label='测试集（真实值）', color='green', marker='o', linewidth=2)

# ETS 预测值
plt.plot(test['Order Date'], ets_forecast,
         label=f'ETS 预测值 (MAE={ets_mae:,.0f})',
         color='orange', linestyle='--', marker='s', linewidth=2)

plt.title('ETS 模型预测 vs 真实值对比', fontsize=14)
plt.xlabel('月份')
plt.ylabel('月销售额')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

for i in range(len(test)):
    month = test.iloc[i]['Order Date'].strftime('%Y-%m')
    actual = test.iloc[i]['Sales']
    pred = ets_forecast.iloc[i]

```

```

diff = pred - actual
pct = (diff / actual) * 100
print(f"month: 真实={actual:,.0f}, 预测={pred:,.0f}, 偏差={diff:,.0f} ({pct:+.1f}%)")

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

# 确保 monthly_sales 是 24 个月完整数据
monthly_sales_clean = monthly_sales[monthly_sales['Order Date'] < '2025-10-01']

sales_series = monthly_sales_clean.set_index('Order Date')['Sales']
sales_diff = sales_series.diff().dropna()

# 画 ACF 和 PACF
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(sales_diff, ax=axes[0], lags=12)
plot_pacf(sales_diff, ax=axes[1], lags=11)
plt.tight_layout()
plt.show()

#ARIMA
from statsmodels.tsa.arima.model import ARIMA

arima_model = ARIMA(train['Sales'], order=(1, 1, 1)).fit()
arima_forecast = arima_model.forecast(6)
arima_mae = mean_absolute_error(test['Sales'], arima_forecast)

print(f"ARIMA(1,1,1) MAE: {arima_mae:,.2f}")
print(f"预测值: {arima_forecast.round(0).tolist()}")

plt.figure(figsize=(14, 5))

# 测试集真实值
plt.plot(test['Order Date'], test['Sales'],
         label='测试集（真实值）', color='blue', marker='o', linewidth=2)

# ARIMA 预测值
plt.plot(test['Order Date'], arima_forecast,
         label=f'ARIMA 预测值 (MAE={arima_mae:,.0f})',
         color='green', linestyle='--', marker='s', linewidth=2)

plt.title('ARIMA 模型预测 vs 真实值对比', fontsize=14)
plt.xlabel('月份')

```

```

plt.ylabel('月销售额')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

for i in range(len(test)):
    month = test.iloc[i]['Order Date'].strftime('%Y-%m')
    actual = test.iloc[i]['Sales']
    pred = arima_forecast.iloc[i]
    diff = pred - actual
    pct = (diff / actual) * 100
    print(f"{month}: 真实={actual:,.0f}, 预测={pred:,.0f}, 偏差={diff:,.0f} ({pct:+.1f}%)")

#品类总体分析
#按品类分析销售额、利润、利润率及销量
category_performance = df.groupby('Category').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Quantity': 'sum',
    'Order ID': 'count'
}).reset_index()

category_performance['Profit_Margin'] = (category_performance['Profit'] / category_performance['Sales']) *
100

category_performance = category_performance.sort_values('Profit_Margin', ascending=False)

print(category_performance[['Category', 'Sales', 'Profit', 'Profit_Margin']].head(10))
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
#销售额排行
sales_rank = category_performance.sort_values('Sales', ascending=True)
axes[0].barh(sales_rank['Category'], sales_rank['Sales'] / 1e6, color='steelblue')
axes[0].set_xlabel('销售额 (百万)')
axes[0].set_title('各品类总销售额排行')

#利润排行
Profit_rank = category_performance.sort_values('Profit', ascending=True)
colors = ['coral' if x < 10 else 'seagreen' for x in Profit_rank['Profit']]
axes[1].barh(Profit_rank['Category'], Profit_rank['Profit'], color=colors)
axes[1].set_xlabel('利润')
axes[1].set_title('各品类利润排行')

plt.tight_layout()

```

```

plt.show()

# 计算各品类销售额占比
category_share = df.groupby('Category')['Sales'].sum() / df['Sales'].sum()

# HHI 指数（赫芬达尔-赫希曼指数）
hhi = (category_share ** 2).sum()
print(f"HHI 指数: {hhi:.4f}")

# 累计占比曲线（帕累托图）
category_sales = df.groupby('Category')['Sales'].sum().sort_values(ascending=False)
category_share = category_sales / category_sales.sum()

cumsum = category_share.cumsum()

print(f"前 1 大品类累计占比: {cumsum.iloc[0]:.1%}")
print(f"前 3 大品类累计占比: {cumsum.iloc[2]:.1%}")
print(f"前 5 大品类累计占比: {cumsum.iloc[4]:.1%}")

fig, ax1 = plt.subplots(figsize=(12, 5))

ax1.bar(category_sales.index, category_sales.values / 1e6, color='steelblue', alpha=0.7)
ax1.set_xlabel('品类')
ax1.set_ylabel('销售额（百万）', color='steelblue')
ax1.tick_params(axis='x', rotation=45)

ax2 = ax1.twinx()
ax2.plot(category_sales.index, cumsum.values * 100, color='red', marker='o', linewidth=2)
ax2.set_ylabel('累计占比（%）', color='red')
ax2.tick_params(axis='y', labelcolor='red')

# 80% 参考线
ax2.axhline(y=80, color='gray', linestyle='--', alpha=0.7, label='80% 线')

plt.title('各品类销售额帕累托图（累计贡献曲线）')
plt.tight_layout()
plt.show()

# 品类折扣敏感度
sensitivity = []
for cat in df['Category'].unique():

```

```

sub = df[df['Category'] == cat]
if len(sub) > 30:
    corr = sub['Discount'].corr(sub['Quantity'])
    sensitivity.append({
        'Category': cat,
        'Correlation': corr,
        'Avg_Discount': sub['Discount'].mean(),
        'Order_Count': len(sub)
    })

sens_df = pd.DataFrame(sensitivity).sort_values('Correlation', ascending=False)
# 2. 画柱状图
fig, ax = plt.subplots(figsize=(12, 5))

colors = ['green' if x > 0.1 else 'orange' if x > -0.1 else 'red' for x in sens_df['Correlation']]
ax.bar(sens_df['Category'], sens_df['Correlation'], color=colors, edgecolor='black')
ax.axhline(y=0, color='black', linestyle='--', linewidth=0.8)
ax.axhline(y=0.1, color='gray', linestyle='--', alpha=0.5, label='弱相关阈值 (0.1)')
ax.axhline(y=-0.1, color='gray', linestyle='--', alpha=0.5, label='弱相关阈值 (-0.1)')

ax.set_xlabel('品类')
ax.set_ylabel('折扣与销量的相关系数')
ax.set_title('各品类折扣敏感度')
ax.legend()
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()

#地区表现分析
#地区汇总
region_summary = df.groupby('Region').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Order ID': 'count',
    'Quantity': 'sum'
}).reset_index()

region_summary['Profit_Margin'] = (region_summary['Profit'] / region_summary['Sales']) * 100
region_summary = region_summary.sort_values('Sales', ascending=False)

print("各地区表现汇总")
print(region_summary.to_string(float_format=lambda x: f'{x:,.2f}'))
#方差分析

```

```

#不同地区销售额、利润是否存在显著差异
from scipy.stats import f_oneway

groups1 = [df[df['Region'] == cat]['Sales'].values for cat in df['Region'].unique()]
f_stat1, p_value1 = f_oneway(*groups1)
print(f"销售额方差分析结果: F 统计量 = {f_stat1:.4f}, p 值 = {p_value1:.10f}")

groups2 = [df[df['Region'] == cat]['Profit'].values for cat in df['Region'].unique()]
f_stat2, p_value2 = f_oneway(*groups2)
print(f"利润方差分析结果: F 统计量 = {f_stat2:.4f}, p 值 = {p_value2:.10f}")

#绘制置信区间
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.pointplot(data=df, x='Region', y='Sales', ax=axes[0],
               capsize=0.2, errorbar='ci', color='steelblue')
axes[0].set_title('各区域销售额均值 ± 95% 置信区间')
axes[0].set_xlabel('区域')
axes[0].set_ylabel('销售额')

sns.pointplot(data=df, x='Region', y='Profit', ax=axes[1],
               capsize=0.2, errorbar='ci', color='coral')
axes[1].set_title('各区域利润均值 ± 95% 置信区间')
axes[1].set_xlabel('区域')
axes[1].set_ylabel('利润')

plt.tight_layout()
plt.show()

#箱线图绘制
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 销售额箱线图
df.boxplot(column='Sales', by='Region', ax=axes[0])
axes[0].set_title('各区域销售额分布')
axes[0].set_xlabel('区域')
axes[0].set_ylabel('销售额')

# 利润箱线图
df.boxplot(column='Profit', by='Region', ax=axes[1])
axes[1].set_title('各区域利润分布')
axes[1].set_xlabel('区域')
axes[1].set_ylabel('利润')

plt.suptitle('')

```

```

plt.tight_layout()
plt.show()

# 区域 × 品类 交叉表（销售额占比）
region_cat = df.groupby(['Region', 'Category'])['Sales'].sum().unstack().fillna(0)
region_cat_pct = region_cat.div(region_cat.sum(axis=1), axis=0) * 100

# 画堆叠柱状图
region_cat_pct.plot(kind='bar', stacked=True, figsize=(16, 8), colormap='tab20')
plt.title('各区域品类销售额占比（区域偏好）')
plt.xlabel('区域')
plt.ylabel('品类占比（%）')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

# 城市级汇总
city_summary = df.groupby('City').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Order ID': 'count',
    'Quantity': 'sum'
}).reset_index()

city_summary['Profit_Margin'] = (city_summary['Profit'] / city_summary['Sales']) * 100
city_summary['Avg_Order_Value'] = city_summary['Sales'] / city_summary['Order ID']
city_summary = city_summary.sort_values('Sales', ascending=False)

city_top10 = city_summary.head(10)

print("城市销售额 TOP10")
print(city_top10[['City', 'Sales', 'Profit_Margin', 'Avg_Order_Value', 'Order ID']].to_string(float_format=lambda x: f'{x:,.2f}'))

# 可视化：城市销售额 + 利润率双轴图
fig, ax1 = plt.subplots(figsize=(14, 6))

# 柱状图：销售额
ax1.bar(city_top10['City'], city_top10['Sales'] / 1e6, color='steelblue', alpha=0.7, label='销售额')
ax1.set_xlabel('城市')
ax1.set_ylabel('销售额（百万）', color='steelblue')
ax1.tick_params(axis='y', labelcolor='steelblue')
ax1.set_title('销售额 Top 10 城市及利润率')

```



```

ax1.set_xticklabels(city_top10['City'], rotation=45)

# 折线图：利润率（双轴）
ax2 = ax1.twinx()
ax2.plot(city_top10['City'], city_top10['Profit_Margin'], color='coral', marker='o', linewidth=2, label='
利润率')
ax2.set_ylabel('利润率（%）', color='coral')
ax2.tick_params(axis='y', labelcolor='coral')

plt.tight_layout()
plt.show()

# 统计每个区域包含多少个城市
region_city_count = df.groupby('Region')['City'].nunique().sort_values(ascending=False)
print("各区域城市数量：")
print(region_city_count)

region_cities = df.groupby('Region')['City'].unique()

fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes = axes.flatten()

# 统一颜色
categories = df['Category'].unique()
colors = plt.cm.tab20.colors[:len(categories)]

for i, region in enumerate(region_cities.index):
    cities = region_cities[region]
    # 城市 × 品类 交叉表（占比）
    city_cat = df[df['City'].isin(cities)].groupby(['City',
'Category'])['Sales'].sum().unstack().fillna(0)
    city_cat_pct = city_cat.div(city_cat.sum(axis=1), axis=0) * 100

    # 画堆叠柱状图
    city_cat_pct.plot(kind='bar', stacked=True, ax=axes[i], color=colors, legend=False)
    axes[i].set_title(f'{region} 区域（{len(cities)} 个城市）')
    axes[i].set_xlabel('城市')
    axes[i].set_ylabel('品类占比（%）')
    axes[i].tick_params(axis='x', rotation=45)
    axes[i].legend().remove()

# 统一图例放在右侧
handles, labels = axes[0].get_legend_handles_labels()

```

```

fig.legend(handles, labels, bbox_to_anchor=(1.02, 0.5), loc='center left', fontsize=10)
plt.tight_layout()
plt.show()

# Top 10 城市 × Top 5 品类
top_cities = df.groupby('City')['Sales'].sum().sort_values(ascending=False).head(10).index
top_cats = df.groupby('Category')['Sales'].sum().sort_values(ascending=False).head(5).index

# 交叉表：城市 × 品类
city_cat = df[df['City'].isin(top_cities)].groupby(['City',
'Category'])['Sales'].sum().unstack().fillna(0)
city_cat_pct = city_cat.div(city_cat.sum(axis=1), axis=0) * 100 # 每个城市内品类占比

# 只保留 Top 5 品类
city_cat_pct = city_cat_pct[top_cats]

# 按偏好排序（让相近的城市排在一起）
from scipy.cluster.hierarchy import linkage, leaves_list

# 聚类排序（让相似的城市相邻）
linkage_matrix = linkage(city_cat_pct, method='average')
order = leaves_list(linkage_matrix)
city_cat_pct = city_cat_pct.iloc[order]

# 画热力图
plt.figure(figsize=(10, 6))
sns.heatmap(city_cat_pct, annot=True, fmt='.1f', cmap='YlOrRd', linewidths=0.5)
plt.title('Top 10 城市 × Top 5 品类 偏好指数 (%)')
plt.xlabel('品类')
plt.ylabel('城市')
plt.tight_layout()
plt.show()

# 汇总所有城市
city_scatter = df.groupby('City').agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()
city_scatter['Profit_Margin'] = (city_scatter['Profit'] / city_scatter['Sales']) * 100

#区分 top10 城市
city_scatter = city_scatter.sort_values('Sales', ascending=False).reset_index(drop=True)
city_scatter['Rank'] = city_scatter.index + 1

```

```
city_scatter['Color'] = city_scatter['Rank'].apply(lambda x: 'red' if x <= 10 else 'steelblue')
```

```
# 画散点图
```

```
fig, ax = plt.subplots(figsize=(14, 8))
```

```
for _, row in city_all.iterrows():
```

```
    plt.scatter(row['Sales'] / 1e6, row['Profit'] / 1e6,
                color=row['Color'], s=80, alpha=0.7,
                edgecolors='black', linewidth=0.5)
```

```
# 标注所有城市（错开方向）
```

```
for i, row in city_scatter.iterrows():
```

```
    x = row['Sales'] / 1e6
```

```
    y = row['Profit'] / 1e6
```

```
    # 根据位置动态调整标签方向
```

```
    if x > avg_sales/1e6 and y > avg_profit/1e6:
```

```
        ha, va = 'left', 'bottom'
```

```
    elif x > avg_sales/1e6 and y <= avg_profit/1e6:
```

```
        ha, va = 'left', 'top'
```

```
    elif x <= avg_sales/1e6 and y > avg_profit/1e6:
```

```
        ha, va = 'right', 'bottom'
```

```
    else:
```

```
        ha, va = 'right', 'top'
```

```
    ax.annotate(row['City'], (x, y), fontsize=8, ha=ha, va=va, alpha=0.8)
```

```
# 均线
```

```
ax.axhline(y=avg_profit/1e6, color='gray', linestyle='--', alpha=0.5, label='利润均值线')
```

```
ax.axvline(x=avg_sales/1e6, color='gray', linestyle='--', alpha=0.5, label='销售额均值线')
```

```
ax.set_xlabel('销售额（百万）')
```

```
ax.set_ylabel('利润（百万）')
```

```
ax.set_title('各城市销售额 vs 利润分布')
```

```
ax.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
#用户行为分析
```

```
# 复购分布
```

```
cust_orders = df.groupby('Customer Name').size().reset_index(name='Order_Count')
```

```
order_dist = cust_orders['Order_Count'].value_counts().sort_index()
```

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(order_dist.index.astype(str), order_dist.values, color='steelblue', edgecolor='black')
```

```
plt.xlabel('客户下单次数')
```

```

plt.ylabel(' 客户数量')
plt.title(' 客户复购分布（下订单次数 vs 客户数）')
for i, v in enumerate(order_dist.values):
    plt.text(i, v + 5, str(v), ha='center', fontsize=10)
plt.tight_layout()
plt.show()

# 复购率
repeat_rate = (cust_orders['Order_Count'] > 1).sum() / len(cust_orders) * 100
print(f"复购率: {repeat_rate:.1f}%")
print(f"单次客户: {(cust_orders['Order_Count'] == 1).sum()}")
print(f"复购客户: {(cust_orders['Order_Count'] > 1).sum()}")

# 标记是否复购
cust_orders['Is_Repeat'] = cust_orders['Order_Count'] > 1
cust_avg = cust_avg.merge(cust_orders[['Customer Name', 'Is_Repeat']], on='Customer Name')

# 分组统计
repeat_avg = cust_avg.groupby('Is_Repeat')['Avg_Order_Value'].mean().reset_index()
repeat_avg['Type'] = repeat_avg['Is_Repeat'].map({True: '复购客户', False: '单次客户'})

plt.figure(figsize=(6, 5))
plt.bar(repeat_avg['Type'], repeat_avg['Avg_Order_Value'], color=['steelblue', 'coral'],
edgecolor='black')
plt.ylabel(' 平均客单价（元）')
plt.title(' 单次 vs 复购客户客单价对比')
for i, v in enumerate(repeat_avg['Avg_Order_Value']):
    plt.text(i, v + 500, f'{v:,.0f}', ha='center', fontsize=10)
plt.tight_layout()
plt.show()

#消费金额分层
customer_value = df.groupby('Customer Name')['Sales'].sum().reset_index()
customer_value.columns = ['Customer', 'Total_Spend']
print(customer_value['Total_Spend'].describe(percentiles=[0.2, 0.4, 0.6, 0.8, 0.9, 0.95, 0.99]))
#按金额分层
bins = [0, 33000, 64000, 120000, 250000, customer_value['Total_Spend'].max()]
labels = ['低价值', '中低价值', '中价值', '高价值', '超高价值']

customer_value['Tier'] = pd.cut(customer_value['Total_Spend'], bins=bins, labels=labels)
tier_counts = customer_value['Tier'].value_counts()
# 查看各层级客户数量
print(customer_value['Tier'].value_counts())

```

```

plt.figure(figsize=(6, 6))
plt.pie(tier_counts, labels=tier_counts.index, autopct='%1.1f%%', colors=['#ff9999', '#66b3ff', '#99ff99'])
plt.title('客户价值分层占比')
plt.tight_layout()
plt.show()

# 各层级销售额贡献（柱状图）
tier_sales = customer_value.groupby('Tier')['Total_Spend'].sum().reset_index()
tier_sales = tier_sales.sort_values('Total_Spend', ascending=False)

plt.figure(figsize=(8, 5))
plt.bar(tier_sales['Tier'], tier_sales['Total_Spend'] / 1e6, color=['#99ff99', '#66b3ff', '#ff9999'])
plt.ylabel('总销售额（百万）')
plt.title('各层级客户销售额贡献')
for i, v in enumerate(tier_sales['Total_Spend'] / 1e6):
    plt.text(i, v + 0.2, f'{v:.1f}M', ha='center')
plt.tight_layout()
plt.show()

# 各层级客单价对比（箱线图）
df_with_tier = df.merge(customer_value[['Customer', 'Tier']], left_on='Customer Name',
right_on='Customer', how='left')

plt.figure(figsize=(8, 5))
df_with_tier.boxplot(column='Sales', by='Tier', grid=False)
plt.title('各层级客户客单价分布')
plt.suptitle('')
plt.xlabel('客户层级')
plt.ylabel('客单价（元）')
plt.tight_layout()
plt.show()

customer_data = df.groupby('Customer Name').agg({
    'Sales': 'sum',
    'Order ID': 'count'
}).reset_index()
customer_data.columns = ['Customer', 'Total_Spend', 'Frequency']

bins = [0, 32000, 64000, 120000, 240000, customer_data['Total_Spend'].max()]
labels = ['低价值', '中低价值', '中价值', '高价值', '超高价值']
customer_data['Tier'] = pd.cut(customer_data['Total_Spend'], bins=bins, labels=labels)

# 颜色映射

```

```

colors = {'低价值': '#ff1cff', '中低价值': '#ffaalc',
          '中价值': '#1cacff', '高价值': '#1cff1c', '超高价值': '#ff1c1c'}

plt.figure(figsize=(12, 7))
for tier in customer_data['Tier'].unique():
    subset = customer_data[customer_data['Tier'] == tier]
    plt.scatter(subset['Total_Spend'], subset['Frequency'],
                label=tier, color=colors.get(tier, 'gray'), alpha=0.6, s=80)

plt.xlabel('总消费金额')
plt.ylabel('购买频次')
plt.title('')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()

# 计算每个客户的平均客单价
customer_avg = df.groupby('Customer Name').agg({
    'Sales': 'mean',
    'Order ID': 'count'
}).rename(columns={'Sales': 'Avg_Order_Value', 'Order ID': 'Frequency'}).reset_index()

customer_avg['Freq_Group'] = customer_avg['Frequency'].apply(
    lambda x: '1次' if x == 1 else '2次' if x == 2 else '3次'
)

# 画箱线图
plt.figure(figsize=(8, 6))
customer_avg.boxplot(column='Avg_Order_Value', by='Freq_Group', grid=False)
plt.title('')
plt.suptitle('')
plt.xlabel('购买频次')
plt.ylabel('平均客单价')
plt.tight_layout()
plt.show()

#支付方式分析
payment_summary = df.groupby('Payment Mode').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Order ID': 'count'
}).reset_index()

```

```

payment_summary['Profit_Margin'] = (payment_summary['Profit'] / payment_summary['Sales']) * 100
payment_summary['Avg_Order_Value'] = payment_summary['Sales'] / payment_summary['Order_ID']
payment_summary = payment_summary.sort_values('Sales', ascending=False)

print("\n 支付方式表现汇总")
print(payment_summary.to_string(float_format=lambda x: f'{x:,.2f}'))

#
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 支付方式销售额占比
axes[0].pie(payment_summary['Sales'], labels=payment_summary['Payment_Mode'],
            autopct='%1.1f%%', startangle=90,
            colors=['#ff9999', '#66b3ff', '#99ff99', '#ffcc99', '#DEDE52'])
axes[0].set_title('支付方式销售额占比')

# 每个客户的平均客单价
cust_avg = df.groupby('Customer_Name')['Unit_Price'].mean().reset_index(name='Avg_Order_Value')

plt.figure(figsize=(10, 4))
plt.boxplot(cust_avg['Avg_Order_Value'], vert=True, patch_artist=True)
plt.ylabel('平均客单价 (元)')
plt.title('客户平均客单价分布')
plt.xticks([1], ['所有客户'])
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()

# 支付方式平均客单价
axes[1].bar(payment_summary['Payment_Mode'], payment_summary['Avg_Order_Value'], color='purple')
axes[1].set_xlabel('支付方式')
axes[1].set_ylabel('平均客单价')
axes[1].set_title('各支付方式平均客单价')
axes[1].grid(axis='y', linestyle='--', alpha=0.3)

# 支付方式利润率
colors = ['seagreen' if x >= payment_summary['Profit_Margin'].mean() else 'coral'
          for x in payment_summary['Profit_Margin']]
axes[2].bar(payment_summary['Payment_Mode'], payment_summary['Profit_Margin'], color=colors)
axes[2].axhline(y=payment_summary['Profit_Margin'].mean(), color='red', linestyle='--',
                label=f'均值: {payment_summary["Profit_Margin"].mean():.1f}%')
axes[2].set_xlabel('支付方式')
axes[2].set_ylabel('利润率 (%)')

```

```
axes[2].set_title('各支付方式利润率')
axes[2].legend()
axes[2].grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```